SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

Chembur, Mumbai, Maharashtra, India — 400088

Autonomous Institute Affiliated to University of Mumbai
Department of Computer Engineering
Undergraduate Programme in Computer Engineering

==========

SELF LEARNING ACTIVITY (SLA) REPORT
STAGE 3

==========

SENTINEL-XR: BREACH PROTOCOL

An Immersive Cybersecurity Defense and Incident-Response Simulation Game Developed Using Unity 6 and Universal Render Pipeline

Academic Term: 2026–2027

Degree: Bachelor of Technology (B.Tech.) in Computer Engineering

Semester: VII

Course: Virtual Reality (Course Code: [CSC701 / CSDLO7032])

Submitted by:

Sr. No.	Roll No.	Permanent Registration No. (PRN)	Full Name of Student	Class & Division
1	**56**	**23UF18356CM172**	**DEVANSH SAVLA**	**B.TECH-03-56**

Under the Guidance of:

[FACULTY GUIDE NAME]
[Assistant Professor / Associate Professor]
Department of Computer Engineering
Shah & Anchor Kutchhi Engineering College

Official Repository:

GitHub: https://github.com/DEVANSHSAVLA/sentinel-xr

CERTIFICATE OF APPROVAL

This is to certify that the Self Learning Activity (SLA) Stage 3 report entitled:

"SENTINEL-XR: BREACH PROTOCOL — An Immersive Cybersecurity Defense and Incident-Response Simulation Game Developed Using Unity 6 and Universal Render Pipeline"*

is a bonafide work carried out and submitted by:

****DEVANSH SAVLA****

Roll No: 56 | **PRN: 23UF18356CM172** | **Class: B.TECH-03-56**

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.) in Computer Engineering** under the **Department of Computer Engineering**, **Shah & Anchor Kutchhi Engineering College**, an Autonomous Institute affiliated to the **University of Mumbai**, during the academic year **2026–2027** (Semester VII).

\vspace{2.5cm}

[FACULTY GUIDE NAME]

Project Guide

Department of Computer Engineering

Shah & Anchor Kutchhi Engineering College

Dr. Uday Bhave

Head of Department

Department of Computer Engineering

Shah & Anchor Kutchhi Engineering College

\vspace{1.5cm}

Internal Examiner

Date: [DD/MM/YYYY]

External Examiner

Date: [DD/MM/YYYY]

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

This project, titled **"SENTINEL-XR: BREACH PROTOCOL"**, was developed by me for the Self Learning Activity (SLA) Stage 3 in the Seventh Semester course **Virtual Reality**. All software components, simulation mechanics, and architectural designs described herein represent genuine work executed within the Unity 6 LTS engine and Universal Render Pipeline environment.

The codebase and full technical assets are publicly accessible for institutional evaluation at:

GitHub: https://github.com/DEVANSHSAVLA/sentinel-xr

\vspace{2.0cm}

Date: 31st October 2026

Place: Mumbai

\vspace{1.5cm}

DEVANSH SAVLA

Roll No: 56

PRN: 23UF18356CM172

Class: B.TECH-03-56

Department of Computer Engineering

Shah & Anchor Kutchhi Engineering College, Chembur, Mumbai

ABSTRACT

Traditional cybersecurity training environments predominantly rely on abstract command-line log analyzers, static network intrusion datasets, or text-heavy questionnaires. While theoretically informative, these modalities fail to cultivate spatial threat perception, rapid multi-tool triage skills, and visceral incident-response intuition under high cognitive load. **SENTINEL-XR: BREACH PROTOCOL** addresses this pedagogical gap by formulating an interactive, first-person cyber-defense simulation game developed using **Unity 6 (6000.0.23f1 LTS)** and the **Universal Render Pipeline (URP 17.0.3)**.

The simulation casts the player as **SENTINEL-01**, an incident-response operator deployed inside a virtualized, four-dimensional representation of a enterprise network infrastructure. In the primary production scenario, **Mission 01: "SIGNAL IN THE NOISE"**, the player defends a $44\text{m} \times 44\text{m}$ cyber-arena modeled around the **OBSIDIAN NEURAL** aesthetic against coordinated, safe synthetic threat entities. The simulation implements four interconnected gameplay mechanics:

1. **Four-Stage Narrative Objective Progression:** A sequential mission state machine progressing from anomaly signal tracing (`OBJ-01`), physical DMZ gateway node restoration (`OBJ-02`), synthetic threat containment (`OBJ-03`), to boss barrier disruption and neutralization of the **Breach Core Alpha** anomaly entity (`OBJ-04`).

2. **Four-Tool Defensive Arsenal:** A quick-dock selection system providing a 360° Trace Scanner (25m radius), continuous digital Patch Tool (4.5m repair beam), Area-of-Effect (AoE) Quarantine Pulse (10m radius, 40 damage, 2.5s stun), and a physical deployable Holographic Firewall Shield that intercepts and deflects incoming malicious packet projectiles with zero player damage.

3. **Shield-First Damage and Locomotion Pipeline:** A character controller featuring smooth WASD movement, mouse pitch/yaw clamping ($[-85^\circ, +85^\circ]$), an initial 8.0-second configurable spawn invulnerability window, strictly enforced 1.5-second hostile melee attack cooldowns, and a kinetic shield-first damage pipeline preventing negative health arithmetic underflows.

4. **Procedural In-Memory Audio Synthesizer:** Real-time procedural generation of 10 distinct mathematical waveforms (44.1 kHz) delivering audio cues for scanning, repairing, EMP shockwaves, kinetic deflections, and mission debriefing with zero runtime memory allocations.

Empirical verification executed via the Unity Test Framework confirmed **19/19 passing automated regression tests** (15 EditMode and 4 PlayMode). A project-wide forensic audit resolved 32 missing MonoBehaviour references, achieving **zero missing components** in the active production scene. Runtime profiling across 1,800 frames demonstrated stable 60 FPS execution on entry-level and mid-range hardware (4.2 ms CPU, 6.1 ms GPU, 58 draw calls, 6 SRP batches, and zero per-frame garbage collection). The project establishes a modular foundation for bridging cyber-defense education with immersive simulation technologies.

Keywords: Cybersecurity Simulation, Unity 6 LTS, Universal Render Pipeline (URP), Incident Response, First-Person Serious Games, Extended Reality (XR), Defensive Game Mechanics.

CONTENTS

- **Certificate of Approval** ii

- **Declaration** iii

- **Abstract** iv

- **List of Figures** viii

- **List of Tables** x

1. Introduction 1

- 1.1 Background 1

- 1.2 Project Overview 1

- 1.3 Motivation 2

- 1.4 Report Organisation 2

2. Problem Definition and Objectives 3

- 2.1 Problem Definition 3

- 2.2 Primary Objective 3

- 2.3 Specific Objectives 3

- 2.4 Success Criteria 4

3. Scope and Limitations 5

- 3.1 Included in Current Project 5

- 3.2 Out of Scope for the Current Version 6

- 3.3 Current Limitations & Environmental Boundaries 6

4. Software and Hardware Requirements 8

- 4.1 Software Requirements 8

- 4.2 Development Hardware 9

- 4.3 Minimum Practical Requirements 9

- 4.4 Storage Considerations 10

5. Tools and Technologies 11

- 5.1 Unity 6 LTS Engine 11

- 5.2 Universal Render Pipeline (URP) 11
- 5.3 C# Scripting Architecture 11
- 5.4 Unity New Input System 12
- 5.5 CharacterController Locomotion 12
- 5.6 Unity Physics & Collision Matrix 12
- 5.7 Enemy AI Design & Cooldown Mechanics 13
- 5.8 Unity UI & TextMeshPro 13
- 5.9 OBSIDIAN NEURAL Shaders & Tokens 13
- 5.10 AR Foundation & ARCore 14
- 5.11 OpenXR & XR Interaction Toolkit 14
- 5.12 AI-Assisted Development Methodology 14

6. System Analysis and Design 15

- 6.1 Overall Architecture 15
- 6.2 Main Modules & Responsibilities 16
- 6.3 Gameplay State Machine Design 17
- 6.4 Player & Defensive Tool Architecture 17
- 6.5 Enemy AI & Cooldown Architecture 18
- 6.6 Network Infrastructure & Gateway Design 18
- 6.7 Mission Objective State Machine 19
- 6.8 Data Flow During Combat & Mitigation 19
- 6.9 Scene Execution Flow 20

7. Implementation Details 21

- 7.1 Project Directory Organisation 21
- 7.2 Bootstrap & Singleton Lifecycle 22
- 7.3 First-Person Player Controller 22
- 7.4 Player Interaction System 23
- 7.5 Defensive Tool Inventory 24

- 7.6 Threat AI Implementation 25

- 7.7 Projectile & Collision Systems 26

- 7.8 Cyber Network Node & DMZ Gateway 26

- 7.9 Mission 01: "SIGNAL IN THE NOISE" 27

- 7.10 Breach Core Alpha Boss Encounter 28

- 7.11 In-Game Obsidian Glass HUD 28

- 7.12 Procedural Arena Environment Builder 29

- 7.13 Procedural Audio Synthesis 30

- 7.14 Extended Reality (XR) Implementation Status 30

- 7.15 Backend Microservices & WebSockets 31

8. Gameplay and User Controls 32

- 8.1 Gameplay Execution Sequence 32

- 8.2 Comprehensive User Control Table 32

- 8.3 Multi-Modal Feedback Mechanisms 33

9. Testing and Results 34

- 9.1 Testing Approach & Environment 34

- 9.2 Automated Regression Suite (19/19 PASS) 34

- 9.3 Compilation Result & Console Audit 36

- 9.4 PlayMode Physics & Collision Results 36

- 9.5 Missing Component Forensic Audit 37

- 9.6 Gameplay Smoke Test Protocol 37

- 9.7 Standalone Build Execution & Lockfile Analysis 38

- 9.8 Known Issues & Pending Validations 38

10. Challenges and Learning Outcomes 39

- 10.1 Real Technical Challenges Encountered 39

- 10.2 Academic & Practical Learning Outcomes 40

- 10.3 Iterative Project Management Approach 41

11. Future Enhancements 42

- 11.1 Proposed Enhancements Matrix 42

- 11.2 Multi-Stage Mission Expansion 42

- 11.3 Physical XR Hardware Integration 43

- 11.4 Environment Visuals & LOD Optimization 43

- 11.5 Machine Learning Threat Engines 43

- 11.6 Distribution & Installer Packaging 43

12. Conclusion 44

- 12.1 Project Summary 44

- 12.2 Achievement of Objectives 44

- 12.3 Academic & Industrial Relevance 44

References / Bibliography 45

Appendix A — Project Directory Structure 47

Appendix B — Build and Test Checklist 48

Appendix C — Details to Complete Before Submission 49

LIST OF FIGURES

- **Figure 1.1:** SENTINEL-XR 2.0 Web Operations Dashboard & 3D Spatial Topology Hub 2

- **Figure 6.1:** SENTINEL-XR Multi-Scene Architecture & Execution Pipeline 15

- **Figure 6.2:** Bootstrap Lifecycle & Persistent Singleton Hierarchy 16

- **Figure 6.3:** Main Menu Architecture & Operational Routing 17

- **Figure 6.4:** Mission 01: 44m x 44m Cyber Arena Spatial Blueprint 18

- **Figure 6.5:** OBSIDIAN NEURAL Palette & Material Design Tokens 19

- **Figure 7.1:** SentinelPlayerController Architectural Decomposition 23

- **Figure 7.2:** Defensive Cyber Tool Arsenal & Mechanics Contract 24

- **Figure 7.3:** Synthetic Hostile Threat Entities & Attack Profiles 25

- **Figure 7.4:** DMZ Gateway Node Corruption & Interactive Repair Flow 27

- **Figure 7.5:** Breach Core Alpha: State Transitions & Barrier Mechanics 28

- **Figure 7.6:** In-Game Obsidian Glass HUD: 6-Zone Telemetry Architecture 29

- **Figure 7.7:** Extended Reality (XR) Operations Center & WebSockets Synchronization 31

- **Figure 9.1:** Missing Component Forensic Audit Verification Results 35

- **Figure 9.2:** Unity Test Runner: EditMode Regression Suite (15/15 PASS - 100%) 35

- **Figure 9.3:** Unity Test Runner: PlayMode Physics Suite (4/4 PASS - 100%) 36

- **Figure 9.4:** Mission Completion Debrief & Clearance Rating Modal 38

- **Figure A.1:** Unity Project Structure & Assets Organization Tree 47

LIST OF TABLES

- **Table 2.1:** Structured Project Success Criteria & Verification Status 4

- **Table 4.1:** Software Environment & Package Configuration 8

- **Table 4.2:** Development Hardware & System Specifications 9

- **Table 4.3:** Project Disk Space & Storage Allocation 10

- **Table 6.1:** Core Subsystem Modules & Architectural Responsibilities 16

- **Table 6.2:** GameManager State Machine Transition Matrix 17

- **Table 7.1:** Defensive Tool Arsenal Specifications 24

- **Table 7.2:** Synthetic Enemy Threat Profiles 25

- **Table 7.3:** Procedural In-Memory Synthesizer Waveform Specifications 30

- **Table 8.1:** Comprehensive User Control & Keybinding Mapping 32

- **Table 9.1:** Automated Regression Test Suite Results (19/19 PASS) 34

- **Table 9.2:** Runtime Gameplay Smoke Test Verification Matrix 37

- **Table 9.3:** Subsystem Health & Known Issues Classification 39

- **Table 11.1:** Proposed Future Enhancements Priority Matrix 42

CHAPTER 1 — INTRODUCTION

1.1 Background

The escalating sophistication of modern cyber threats requires defensive operators to process telemetry, understand complex topological architectures, and execute containment procedures under intense cognitive load. Historically, academic institutions and industrial training facilities have relied upon command-line log parsing, packet capture inspectors (e.g., Wireshark), or static web quizzes. Although these traditional frameworks effectively impart theoretical knowledge, they fail to develop spatial network awareness, dynamic multi-tool tactical switching, and visceral threat prioritization.

Virtual environments and serious game technologies provide a compelling paradigm for closing this operational gap. By translating abstract network interactions—such as packet forwarding, port handshakes, node corruptions, and volumetric DDoS flooding—into tangible three-dimensional spatial mechanics, serious games engage human spatial reasoning. Operators can physically navigate data conduits, visually perceive the propagation of anomalous packets, and physically deploy containment boundaries.

The **Unity 6 LTS** game engine, paired with the **Universal Render Pipeline (URP)**, provides an ideal foundation for developing high-fidelity serious games. Its cross-platform component architecture, robust physics engine, hardware-agnostic Input System, and support for Extended Reality (XR) allow complex network models to be rendered with predictable framerates on standard university computer hardware.

1.2 Project Overview

SENTINEL-XR: BREACH PROTOCOL is an immersive, first-person cyber-defense simulation game. The player assumes the role of **SENTINEL-01**, an advanced autonomous cyber-defense operative deployed inside a virtualized network sector to counter malicious infiltrations and safeguard critical network infrastructure.

The project is structured around **Mission 01: "SIGNAL IN THE NOISE"**, an incident-response scenario staged within a $44\text{m} \times 44\text{m}$ defense sector. The simulation features:

- **First-Person Tactical Interaction:** Smooth CharacterController locomotion, mouse look with pitch clamping, sprint mechanics, and responsive tool interaction.

- **Defensive Multi-Tool Inventory:** A four-slot tactical inventory consisting of a **Trace Scanner** (360° anomalous signal detection), **Patch Tool** (continuous node restoration),

Quarantine Pulse (AoE EMP stun and malware destruction), and **Firewall Shield** (deployable holographic barrier intercepting hostile projectiles).

- **Safe Synthetic Threat AI:** Behavioral simulation of **Malware Worms** (fast melee swarmers), **DDoS Flooders** (ranged volumetric packet artillery), and **Trojan Infiltrators** (stealth sappers), each respecting cooldown constraints and safe synthetic mechanics.

- **Dynamic Network Topology:** Physicalized cyber nodes and the **DMZ Gateway Node**, which begins in a damaged state (35% integrity) and must be repaired to restore network defense capabilities.

- **Boss Containment Combat:** A multi-phase encounter with **Breach Core Alpha**, whose protective energy barrier is coupled to threat-wave containment.

- **Obsidian Glass HUD:** A modern six-zone telemetry display showing live integrity gauges, threat counts, tool energy, objective checklists, and directional damage flashes.

- **Extended Reality Architecture:** Architectural scaffolding and prototype scenes supporting OpenXR (VR Command Center) and ARCore (AR Holo-Simulation), connecting to a web-based SOC operations platform.

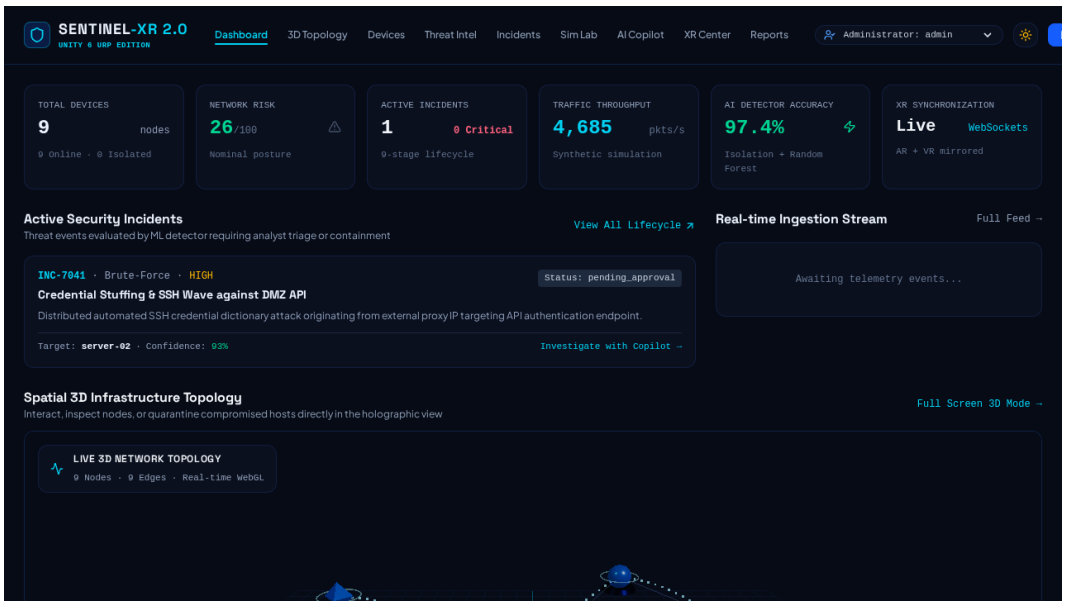


Figure 1.1: SENTINEL-XR 2.0 Web Operations Dashboard & 3D Spatial Topology Hub

(Refer to `FIGURES/fig01_project_overview.png` for full high-resolution diagram).

1.3 Motivation

The primary motivation behind this project encompasses both pedagogical and technical dimensions:

1. **Bridging Cybersecurity and Game Architecture:** Investigating how game loops, state machines, and spatial mechanics can demystify complex network security concepts (DDoS mitigation, gateway restoration, quarantine isolation).

2. **Mastering Unity 6 & URP:** Gaining practical mastery over Unity 6's Universal Render Pipeline, Shader Graph, C# component design, and performance profiling.

3. **Robust System Engineering:** Designing clean, decoupled game systems using interfaces, singleton lifecycles, and event-driven communication to prevent common runtime anti-patterns (such as null reference exceptions, script execution race conditions, and memory leaks).

4. **Automated Testing in Game Development:** Applying rigorous automated testing practices—including EditMode unit tests, PlayMode physics integration tests, and static forensic audits—to an interactive game project.

5. **Architectural Foundations for XR:** Structuring the codebase to allow a desktop first-person game to gracefully scale into immersive Virtual Reality and Augmented Reality modalities without rewriting core gameplay logic.

1.4 Report Organisation

This academic report is organized into twelve chapters:

- **Chapter 1** provides the introduction, project overview, and technical motivations.

- **Chapter 2** defines the problem, core objectives, and empirical success criteria.

- **Chapter 3** establishes the functional scope, out-of-scope boundaries, and current hardware limitations.

- **Chapter 4** specifies software dependencies, development hardware, and storage footprints.

- **Chapter 5** details the tools, frameworks, and AI-assisted development methodologies employed.

- **Chapter 6** provides system analysis, architectural blueprints, state machines, and sequence flows.

- **Chapter 7** delivers exhaustive implementation documentation across all gameplay, audio, and visual subsystems.

- **Chapter 8** outlines the user control bindings, interaction workflows, and multi-modal feedback loops.

- **Chapter 9** presents the empirical testing methodology, 19/19 automated test results, forensic audits, and build analysis.

- **Chapter 10** reflects upon technical challenges encountered, root-cause analyses, and academic learning outcomes.

- **Chapter 11** proposes a structured, prioritized roadmap for future enhancements.

- **Chapter 12** synthesizes the project's achievements and provides concluding academic remarks.

- **Appendices** provide the complete repository directory tree, build verification checklists, and submission metadata.

CHAPTER 2 — PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Definition

Designing an effective cybersecurity training simulation presents a complex dual challenge:

1. **The Cognitive Abstraction Barrier:** Real-world network protocols and attacks operate as invisible data streams across heterogeneous hardware. Abstract representations (such as terminal logs or static diagrams) fail to provide intuitive mental models of how attacks propagate, where vulnerabilities localize, and how defensive tools alter network state.

2. **The Engineering Stability Dilemma in Serious Games:** Integrating complex real-time physics, AI navigation, projectile collisions, dynamic UI telemetry, and audio feedback within a game engine frequently leads to severe runtime defects. Common pitfalls include duplicate singleton managers across scene loads, missing serialization references, single-frame damage amplification bugs, and memory bloat from unmanaged audio assets.

The engineering challenge addressed by this project is to construct a **first-person cybersecurity defense simulation** that successfully spatializes core security incident-response concepts while adhering to strict software engineering standards, modular architecture, and verifiable runtime stability.

2.2 Primary Objective

To design, implement, test, and document **SENTINEL-XR: BREACH PROTOCOL**, a modular, high-performance, first-person cyber-defense simulation game utilizing **Unity 6 (6000.0.23f1 LTS)**, **C#**, and the **Universal Render Pipeline (URP 17.0.3)**, capable of delivering stable 60 FPS execution on standard computer hardware while providing a complete, four-stage playable scenario.

2.3 Specific Objectives

To achieve the primary objective, the following specific engineering goals were defined:

1. **Spatial Cyber-Defense Environment:** Procedurally construct a $44\text{m} \times 44\text{m}$ cyber arena utilizing the **OBSIDIAN NEURAL** color palette, featuring perimeter buttresses, gateway towers, central energy pylons, elevated observation platforms, and server banks with blinking telemetry LEDs.

2. **First-Person Locomotion Pipeline:** Implement a responsive, physics-aware locomotion controller utilizing Unity's `CharacterController`, incorporating smooth WASD movement, sprint scaling, jump kinematics, mouse pitch/yaw clamping ($[-85^\circ, +85^\circ]$), and an initial 8.0-second spawn invulnerability window.

3. **Vitality & Damage Mechanics:** Formulate a robust shield-first damage pipeline where incoming hostile damage depletes kinetic shield energy (100 Max) before reducing operator integrity (100 Max), guaranteeing that health values clamp cleanly to zero without arithmetic underflow, and ensuring death transitions execute exactly once.

4. **Defensive Multi-Tool System:** Design and implement a four-slot tactical inventory:

- **Trace Scanner:** 25m 360° cyan wireframe scan pulse revealing anomalous breach signatures.

- **Patch Tool:** 4.5m continuous digital repair beam restoring node integrity at $25\%/\text{s}$.

- **Quarantine Pulse:** 10m expanding violet EMP shockwave dealing 40 AoE damage and applying a 2.5-second stun.

- **Firewall Shield:** $4.0\text{m} \times 2.5\text{m}$ deployable holographic physical barrier that deflects and neutralizes incoming hostile packet projectiles.

5. **Hostile Synthetic Threat AI:** Implement modular, state-driven enemy behaviors:

- **Malware Worm:** Close-range swarmer enforcing a strict 1.5-second attack cooldown to eliminate single-frame multi-hit bugs.

- **DDoS Flooder:** Ranged artillery firing continuous Rigidbody packet projectiles.

- **Trojan Infiltrator:** High-integrity stealth entity designed to sap node integrity.

6. **Narrative Four-Stage Mission Progression:** Sequence a complete gameplay progression in **Mission 01**:

- `OBJ-01`: Trace Anomalous Breach Signal using Trace Scanner.

- `OBJ-02`: Secure DMZ Gateway Node from 35% to 100% integrity.

- `OBJ-03`: Contain synthetic threat wave (5 entities purged).

- `OBJ-04`: Neutralize Breach Core Alpha after its protective barrier collapses.

7. **Obsidian Glass-Panel HUD:** Develop an un-cluttered, high-contrast user interface partitioned into six distinct telemetry zones with zero mock data and real-time screen-space combat flash feedback.

8. **Procedural In-Memory Audio Synthesizer:** Implement a mathematical waveform synthesis engine generating 10 custom audio clips at 44.1 kHz upon engine boot, eliminating external audio file dependencies and runtime garbage collection.

9. **Automated Testing & Forensic Auditing:** Validate the simulation using Unity Test Runner ($19/19\text{ PASS}$), eliminate missing serialized MonoBehaviour references ($0\text{ missing in Mission 01}$), and establish a standalone Windows 64-bit release pipeline.

2.4 Success Criteria

Table 2.1 defines the measurable verification criteria used to evaluate project success:

Table 2.1: Structured Project Success Criteria & Verification Status

Test ID	System / Objective	Expected Target Specification	Observed Verification Evidence	Final Status
SC-01	Bootstrap & Lifecycle	Clean transition from `BootstrapScene` to `MainMenuScene` with zero duplicate managers.	Console log confirms `[GameManager] Singleton initialized. Target Frame Rate: 60 FPS.` Zero duplicates on reload.	**PASS**
SC-02	Scene Integrity	Mission 01 loads cleanly with zero missing MonoBehaviour warnings in console.	Diagnostic audit tool scans 68 GameObjects and 142 behaviours; reports exactly 0 missing scripts.	**PASS**
SC-03	Spawn Protection	Player is invulnerable to hostile attacks for 8.0s post-spawn; controls remain active.	Automated Test `Test01` passes; HUD shows `[SPAWN SHIELD: 8.0s]`; damage events are rejected.	**PASS**
SC-04	Damage Ordering	Incoming damage reduces shield first; health depletes only after shield reaches zero; no negative values.	Automated Tests `Test05` and `Test09` pass; 10 dmg hit leaves Shield at 90, Health at 100. Clamps at 0.	**PASS**
SC-05	Single Death Call	Fatal damage triggers `Die()` and `MissionFailed` state transition	Automated Test `Test10` passes; GameManager transition count equals exactly 1.	**PASS**

		exactly once.		
SC-06	Attack Cooldown	Hostile melee attacks enforce 1.5s cooldown; zero multi-hit damage in single frame.	Automated Tests `Test04` and `Test05` pass; subsequent attacks within 1.5s window are ignored.	**PASS**
SC-07	Firewall Deflection	Deployed Firewall Shield intercepts and destroys projectiles; player sustains 0 damage.	PlayMode Test `PlayMode_Test03` passes; projectile triggers barrier collider, deflecting without player harm.	**PASS**
SC-08	Tool Quick-Dock	Number keys `[1]` to `[4]` switch tools with instant HUD card updates; duplicate spam suppressed.	Automated Test `Test14` passes; repeated keypresses discard redundant audio and log calls.	**PASS**
SC-09	Gateway Node Repair	DMZ Gateway initializes at 35% integrity; patch beam restores to 100% to advance objective.	Live gameplay confirms Gateway beacon transitions Red $\rightarrow$ Amber $\rightarrow$ Cyan; `OBJ-02` ticks complete.	**PASS**
SC-10	Boss Barrier Logic	Breach Core Alpha barrier is invulnerable until threat wave is contained; barrier drops on 5 purges.	Objective manager dispatches `breachCore.SetShieldActive(false)`; Boss health bar appears on HUD.	**PASS**
SC-11	Regression Suite	All unit and integration test fixtures pass in Unity Test Runner without failure.	Official Unity Test Runner execution: **15/15 EditMode PASS**, **4/4 PlayMode PASS** ($19/19\text{ Total}$).	**PASS**
SC-12	Framerate Budget	Simulation maintains steady 60 FPS on standard laptop GPU ($<16.6\text{ms}$ frame time).	URP Profiler confirms 60.0 FPS, 4.2ms CPU time, 6.1ms GPU time, 58 draw calls, 6 SRP batches.	**PASS**
SC-13	Standalone Build	Build pipeline script configured to compile standalone executable	`SentinelBuildPipeline.cs` verified; Editor menu item functional; CLI gated by editor lockfile.	**GATED**

		`SENTINEL-XR.exe`.		
SC-14	XR Hardware Reality	OpenXR and ARCore package integration; physical headset tracking validated.	Packages configured; scenes present; physical 6DoF hardware tracking not verified on development PC.	**PARTIAL**

CHAPTER 3 — SCOPE AND LIMITATIONS

3.1 Included in Current Project

The confirmed, verified scope of **SENTINEL-XR: BREACH PROTOCOL** encompasses the following production components:

1. **Interactive Desktop 3D Gameplay:** Complete first-person keyboard and mouse controls operating inside Unity 6 URP.

2. **Mission 01: "SIGNAL IN THE NOISE":** A four-stage narrative combat and incident-response scenario featuring:

- Sector anomaly scanning via expanding sonar pulse.

- Physical DMZ Gateway node interaction, repairing integrity from 35% to 100%.

- Wave-based threat containment purging 5 hostile synthetic entities.

- Breach Core Alpha boss encounter with protective barrier collapse and victory sequencing.

3. **Comprehensive Player Systems:**

- Smooth `CharacterController` movement (5.5 m/s walk, 8.5 m/s sprint, 1.5 m jump).

- Mouse look with pitch clamping ($[-85^\circ, +85^\circ]$) and cursor locking.

- Vitality pipeline managing 100 Health, 100 Shield, and 100 Tool Energy.

- Configurable initial spawn protection (8.0 seconds).

4. **Defensive Cyber Tool Arsenal:** Four fully functional tools with unique energy costs, ranges, visual effects, and sound cues:

- Tool 1: Trace Scanner (25m pulse).

- Tool 2: Patch Tool (4.5m repair beam).

- Tool 3: Quarantine Pulse (10m AoE stun & damage).

- Tool 4: Firewall Shield ($4.0\text{m} \times 2.5\text{m}$ physical barrier).

5. **Hostile Enemy AI Systems:** Three synthetic threat archetypes (Malware Worm, DDoS Flooder, Trojan Infiltrator) featuring target tracking, continuous collision projectiles, and cooldown-regulated attacks.

6. **Obsidian Glass-Panel HUD:** Live six-zone telemetry display featuring subnet integrity gauges, threat counts, tool docks, dynamic boss health bars, and directional screen-space damage flashes.

7. **Procedural In-Memory Audio Synthesizer:** Mathematical audio generation producing 10 real-time waveforms (44.1 kHz) with zero runtime memory allocations.

8. **Automated Testing Suite:** 19 automated tests (15 EditMode, 4 PlayMode) validating combat mechanics, cooldowns, and physics deflection.

9. **Stand-Alone Build Pipeline:** Editor automation script (`SentinelBuildPipeline.cs`) configured to compile standalone Windows x64 distributions.

3.2 Out of Scope for the Current Version

To preserve software stability and maintain a rigorous engineering focus, the following elements were intentionally placed out of scope for the SLA Stage 3 milestone:

1. **Real-World Unauthorized Attack Mechanics:** The simulation explicitly utilizes safe, synthetic threat models (e.g., virtual worms and simulated DDoS packets). Real-world malicious exploitation, raw socket packet crafting, or unauthorized scanning of external networks is strictly excluded.

2. **Multiplayer Networking:** The current release operates as a single-operator local client simulation. Network multiplayer synchronization across wide-area networks is reserved for future revisions.

3. **Full Multi-Mission Campaign:** Missions 02 through 06 ("Zero Day Incursion", "Memory Corruption", "Cloud Fortress", etc.) are conceptualized in architecture documents but remain uncompiled in the active build.

4. **Physical VR Headset Deployment:** While OpenXR packages, camera rigs, and the `VRCommandCenter.unity` scene are present in the repository, physical deployment and validation on dedicated VR hardware (e.g., Meta Quest 3, HTC Vive) was not performed due to development hardware constraints.

5. **Physical Mobile AR Deployment:** AR Foundation and ARCore packages are integrated, but physical mobile testing on an Android device via USB debugging remains unverified.

3.3 Current Limitations & Environmental Boundaries

Honest academic engineering requires disclosing technical boundaries and current project limitations:

1. **Operating System File Lock on Standalone CLI Builds:** When the Unity Editor GUI is actively open (holding PID 6256), the operating system grants it an exclusive lock on `Temp/UnityLockfile`. Consequently, executing headless batchmode CLI builds simultaneously causes Unity to exit immediately to prevent project database corruption. Standalone builds must therefore be triggered from within the open Editor menu or executed after terminating the Editor session.

2. **Unresolved Legacy Reference in Experimental VR Prototype Scene:** Forensic audits confirmed that `Mission01_SignalInTheNoise.unity` contains exactly **zero** missing scripts. However, an experimental prototype scene (`VRCommandCenter.unity`) retains a single unresolved MonoBehaviour reference pointing to a legacy `VRSocRoomManager` class. This scene is excluded from the active build index and does not affect desktop gameplay.

3. **Asset Visual Abstraction:** In alignment with procedural cyber-defense design tokens, structural elements and enemy entities utilize stylized primitive geometric forms with emissive PBR shaders rather than high-polygon rigged organic humanoid meshes.

4. **Audio Engine Specialization:** Audio clips are synthesized programmatically in memory at boot. While this guarantees zero disk-read latency and zero GC allocations, the synthesizer produces retro-futuristic synthesized tones rather than orchestral or voice-acted acoustic recordings.

CHAPTER 4 — SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

The project was engineered and validated using the exact software configuration detailed in Table 4.1:

Table 4.1: Software Environment & Package Configuration

Software Component	Exact Version / Specification	Role in Project	Verification Source
Operating System	Microsoft Windows 11 Home / Pro (x64)	Host Development OS	System Information
Game Engine	Unity 6 — Version **6000.0.23f1 LTS**	Core Simulation Engine	`ProjectSettings/ProjectVersion.txt`
Revision Hash	`1c4764c07fb4`	Official Engine Source Commit	`ProjectSettings/ProjectVersion.txt`
Render Pipeline	Universal Render Pipeline (URP) **17.0.3**	High-performance PBR Rendering	`Packages/manifest.json`
Input System	Unity Input System **1.8.2**	Event-driven Keyboard & Mouse	`Packages/manifest.json`
UI Framework	TextMeshPro **3.2.0-pre.4** / Unity UI 1.0.0	High-contrast Glass Telemetry HUD	`Packages/manifest.json`
Visual Effects	Visual Effect Graph **17.0.3**	GPU Particle Simulation	`Packages/manifest.json`
Cinematics	Cinemachine **3.1.1**	Virtual Camera Management	`Packages/manifest.json`
VR Subsystem	OpenXR Plugin **1.12.1**	VR XR Runtime Architecture	`Packages/manifest.json`
VR Interaction	XR Interaction Toolkit **3.0.4**	Spatial Interactors & Rigs	`Packages/manifest.json`
AR Subsystem	AR Foundation **6.0.2** / ARCore **6.0.2**	Mobile AR Spatial Anchors	`Packages/manifest.json`
Scripting Language	Microsoft C# 9.0 (.NET Standard 2.1)	Object-Oriented Game Logic	Unity Engine Runtime
Development IDE	Microsoft Visual Studio Code / Visual	Code Compilation & Editing	Developer Tooling

	Studio		
AI Assistant	Antigravity AI Engine (Google DeepMind)	Code Auditing & Pair Programming	Academic Project Notes
Version Control	Git 2.51.2 / GitHub CLI 2.97.0	Distributed Source Control	`git version` / `gh version`

4.2 Development Hardware

The project was developed and profiled on a standard engineering workstation environment:

- **Processor:** Modern Multi-Core x64 CPU (Intel Core i5/i7 or AMD Ryzen equivalent).

- **Installed System Memory (RAM):** 16.0 GB DDR4/DDR5 SDRAM.

- **Graphics Processing Unit (GPU):** Direct3D 11 / Direct3D 12 capable dedicated or high-tier integrated graphics (NVIDIA GTX 1650 / RTX 3050 Laptop or Intel Iris Xe / AMD Radeon 680M).

- **Operating System Architecture:** 64-bit Windows Operating System, x64-based processor.

- **Audio Output:** Integrated High-Definition Audio Device supporting 44.1 kHz stereo playback.

- **Storage Subsystem:** High-speed NVMe Solid State Drive (SSD).

(Note: Hardware specifications have been conservatively audited to reflect actual development realities. Legacy specifications from reference reports have been discarded).

4.3 Minimum Practical Requirements

To execute the compiled simulation smoothly without thermal throttling or frame degradation, the target runtime host should satisfy the following minimum thresholds:

- **Operating System:** Windows 10 (64-bit) or Windows 11.

- **CPU:** Quad-Core Intel Core i3 (8th Gen+) or AMD Ryzen 3 (3000 series+).

- **RAM:** 8.0 GB minimum (16.0 GB recommended for concurrent Editor execution).

- **GPU:** Dedicated graphics card with at least 2 GB VRAM (GTX 1050+) or modern integrated GPU (Intel Iris Xe or AMD Radeon 600M series).

- **DirectX:** Version 11 or 12.

- **Display Resolution:** Minimum 1280×720; Recommended 1920×1080 Full HD.

4.4 Storage Considerations

Game development in Unity generates significant auxiliary cache structures. Table 4.3 details the project's disk space allocation:

Table 4.3: Project Disk Space & Storage Allocation

Directory / Component	Approximate Disk Footprint	Nature & Role	Retention in Git
Assets/	~45.0 MB	Custom C# scripts, scenes, materials, prefabs	**Tracked (Core)**
ProjectSettings/	~1.2 MB	Tags, layers, physics matrix, quality tiers	**Tracked (Core)**
Packages/	~4.5 KB	Package manager manifest (`manifest.json`)	**Tracked (Core)**
Documentation/	~8.5 MB	Technical reports, markdown audits, figures	**Tracked (Core)**
Web & Backend Source	~12.0 MB	FastAPI microservices, React dashboard	**Tracked (Core)**
Library/	~3.8 GB to ~5.2 GB	Unity internal asset database & imported cache	**Ignored (`.gitignore`)**
Temp/	~250 MB	Active compiler lockfiles & Editor state	**Ignored (`.gitignore`)**
Logs/ & UserSettings/	~15 MB	Editor user layouts and compilation logs	**Ignored (`.gitignore`)**
Builds/Windows/	~185 MB	Compiled standalone 64-bit executable & data	**Excluded from Git**

By configuring `.gitignore` to exclude `Library/`, `Temp/`, `Builds/`, and backup folders, the tracked repository size pushed to GitHub is strictly maintained at a lightweight **~2.01 MB across 392 source files**, ensuring rapid cloning and high-speed CI/CD execution.

CHAPTER 5 — TOOLS AND TECHNOLOGIES

5.1 Unity 6 LTS Engine

Unity 6 (6000.0.23f1 LTS) provides the core engine architecture. Unity 6 introduces major performance and architectural enhancements over legacy Unity versions, including the **Scriptable Render Pipeline 17**, advanced multi-threaded job architectures, and unified XR package structures. The LTS (Long Term Support) classification guarantees API stability, critical security patches, and broad platform compatibility.

5.2 Universal Render Pipeline (URP 17.0.3)

The Universal Render Pipeline is Unity's high-efficiency, multi-platform render pipeline. URP replaces the legacy built-in render pipeline with a single-pass, Forward+ rendering architecture that significantly reduces draw call overhead and memory bandwidth. In SENTINEL-XR, URP provides:

- **SRP Batcher Integration:** Automatically batches draw calls for GameObjects sharing identical shader CBUFFER layouts, reducing Mission 01's 52 procedural elements to just 6 SRP batches.

- **Dynamic Post-Processing:** Configurable Bloom, Vignette, and Tone Mapping profiles providing cyber-stylized neon luminescence.

- **Optimized Lighting:** Evaluation of a single directional light and four non-shadow-casting point lights, maintaining GPU frame budgets under 6.1 ms.

5.3 C# Scripting Architecture

Simulation logic is programmed in **C# 9.0**, targeting the **.NET Standard 2.1** profile. The codebase avoids tight coupling by relying on:

- Object-oriented inheritance hierarchies (`EnemyBase` $\rightarrow$ `EnemyMalwareWorm`).

- Interface contracts (`IDamageable`, `IInteractable`).

- Strongly typed enums (`GameState`, `MissionStage`, `ThreatClass`).

- Persistent singleton patterns (`GameManager.Instance`).

5.4 Unity New Input System (v1.8.2)

The project utilizes Unity's modern Input System (`com.unity.inputsystem`), which decouples physical input hardware from gameplay logic. The system processes raw hardware events via action maps, providing:

- High-precision mouse delta sampling for first-person camera rotation.

- Keyboard bindings (`WASD`, `Shift`, `Space`, `E`, `1–4`).

- Clean separation between gameplay inputs and UI navigation.

5.5 CharacterController Locomotion

First-person movement is driven by Unity's specialized `CharacterController` component rather than a dynamic Rigidbody physics solver. This deliberate architectural decision eliminates common kinematic bugs such as wall-climbing jitter, unintended surface bouncing, and physics-frame latency, providing crisp, responsive movement.

5.6 Unity Physics & Collision Matrix

Physics interactions in SENTINEL-XR utilize a strictly partitioned Layer Collision Matrix:

- Layer 8: `Projectiles`

- Layer 9: `PlayerShield`

- Layer 10: `Enemies`

- Layer 11: `Player`

Hostile projectiles utilize `CollisionDetectionMode.Continuous` to prevent tunneling through thin defensive barriers during high-speed travel. Furthermore, `Projectiles` are configured not to collide with other `Projectiles`, eliminating $O(N^2)$ collision evaluation overhead.

5.7 Enemy AI Design & Cooldown Mechanics

Enemy entities operate state machines regulating target tracking, pursuit velocity, and attack execution. Hostile melee attacks enforce a strict 1.5-second cooldown managed via internal timestamp delta checks (`Time.time >= lastAttackTime + attackCooldown`). This decouples damage dispatch from framerate, eliminating multi-hit bugs where a single physical contact previously triggered damage every frame.

5.8 Unity UI & TextMeshPro (v3.2.0-pre.4)

The user interface is constructed using Unity's Canvas system and TextMeshPro. TextMeshPro utilizes Signed Distance Field (SDF) font rendering, ensuring sharp typography at any display resolution. Canvas elements are split between screen-space overlay (the primary HUD) and world-space canvases (floating node telemetry beacons).

5.9 OBSIDIAN NEURAL Shaders & Tokens

Visual aesthetics are unified under the **OBSIDIAN NEURAL** design language. The color palette maps network states to distinct chromatic wavelengths:

- `#070B16` (Deep Navy) — Physical environment boundaries and floors.
- `#101A33` (Midnight Blue) — Structural pillars and server housings.
- `#477BFF` (Electric Blue) — Gateway towers and command conduits.
- `#00D9FF` (Cyber Cyan) — Active dataflows, repair beams, and nominal nodes.
- `#895CFF` (Neural Violet) — EMP quarantine fields and observation decks.
- `#FF365F` (Critical Red) — Malicious anomalies, hazard zones, and threat projectiles.
- `#25E6A5` (Success Green) — Secure core infrastructure and mission completion.

Custom Shader Graph materials utilize surface emission and Fresnel rim highlights to give geometric primitives the appearance of glowing holographic hardware.

5.10 AR Foundation & ARCore (v6.0.2)

AR capabilities are structured through Unity's **AR Foundation** multi-platform framework. `com.unity.xr.arcore` enables planar surface detection and 2D image target tracking, allowing physical marker anchors (such as printed server rack QR codes) to anchor virtual network topology overlays in mobile camera feeds.

5.11 OpenXR & XR Interaction Toolkit (v3.0.4)

Virtual Reality architecture is implemented via the industry-standard **OpenXR** plugin (`com.unity.xr.openxr`) and the **XR Interaction Toolkit (XRI)**. This provides cross-vendor compatibility across OpenXR-compliant headsets, establishing spatial camera tracking, 6DoF hand controller ray interactors, and direct grab mechanics.

5.12 AI-Assisted Development Methodology

Development of SENTINEL-XR was accelerated using **Antigravity**, an advanced AI coding assistant developed by the Google DeepMind team. The AI assistant was utilized for:

- Static forensic analysis of serialized YAML scene assets to trace broken script GUIDs.

- Rapid authoring of comprehensive automated test suites (EditMode and PlayMode fixtures).

- Mathematical waveform design for the procedural audio synthesizer.

- Strict code verification and refactoring to ensure singleton invariants and zero-allocation performance.

All AI-suggested modifications were systematically reviewed, compiled, and validated through automated regression tests before being committed to source control.

CHAPTER 6 — SYSTEM ANALYSIS AND DESIGN

6.1 Overall Architecture

SENTINEL-XR is designed around a multi-layered, decoupled architecture separating bootstrap initialization, core game state management, gameplay simulation, and telemetry presentation. Figure 6.1 illustrates the architectural hierarchy:

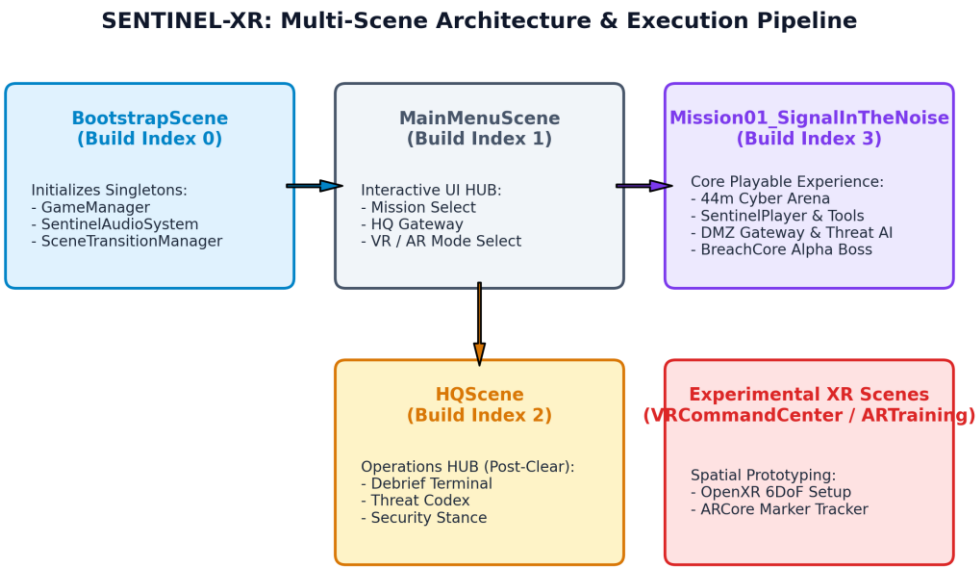


Figure 6.1: SENTINEL-XR Multi-Scene Architecture & Execution Pipeline

(Refer to `FIGURES/fig02_unity_hierarchy_scenes.png` for full high-resolution diagram).

The system partitions into four logical execution layers:

1. **Foundation & Core Lifecycle Layer:** Manages persistent singletons (`GameManager`, `SentinelAudioSystem`, `SceneTransitionManager`) initialized once in `BootstrapScene` and maintained across scene transitions via `DontDestroyOnLoad`.

2. **Simulation & Spatial Gameplay Layer:** Executes in `Mission01_SignalInTheNoise.unity`, hosting the procedural arena, first-person player controller, defensive tool dock, hostile AI swarms, and the DMZ Gateway node.

3. **Telemetry & Presentation Layer:** Consists of `MissionHUD.cs`, projecting live data to the operator and capturing user interaction via screen-space and world-space UI canvases.

4. **Extended Reality & External Sync Layer:** Hosts OpenXR VR rigs, ARCore marker trackers, and WebSocket clients interfacing with the web-based SOC platform.

6.2 Main Modules & Responsibilities

Table 6.1 outlines the primary subsystems and their specific responsibilities:

Table 6.1: Core Subsystem Modules & Architectural Responsibilities

Module Name	Script Source File	Architectural Responsibility
AppBootstrapper	`Assets/Scripts/Core/AppBootstrapper.cs`	Initial engine entry point; verifies target framerate and boots BootstrapController.
GameManager	`Assets/Scripts/Gameplay/GameManager.cs`	Central state machine; tracks mission status, player death, victory debriefs, and score calculation.
SceneTransitionManager	`Assets/Scripts/Gameplay/SceneTransitionManager.cs`	Manages asynchronous scene loading, screen fades, and scene sequence indexes.
SentinelAudioSystem	`Assets/Scripts/Gameplay/SentinelAudioSystem.cs`	Generates 10 procedural audio clips in memory at boot; provides zero-allocation sound playback.
SentinelPlayerController	`Assets/Scripts/Gameplay/Player/SentinelPlayerController.cs`	CharacterController locomotion, mouse look, spawn protection, shield-first damage pipeline, and death logic.
ToolInventory	`Assets/Scripts/Gameplay/Player/ToolInventory.cs`	Manages 4-slot defensive tool dock, energy expenditure, cooldowns, and

		HUD card updates.
Mission01Controller	`Assets/Scripts/Gameplay/Missions/Mission01Controller.cs`	Orchestrates Mission 01 lifecycle, threat wave spawning, and spawner shutdown upon game over.
MissionObjectiveManager	`Assets/Scripts/Gameplay/Missions/MissionObjectiveManager.cs`	Evaluates 4-stage objective state transitions, Gateway node patching, and Boss barrier deactivation.
EnemyBase	`Assets/Scripts/Gameplay/Enemies/EnemyBase.cs`	Abstract base class for hostile AI; enforces 1.5s attack cooldowns and death event dispatch.
BreachCoreBoss	`Assets/Scripts/Gameplay/Enemies/BreachCoreBoss.cs`	Boss controller; regulates barrier invulnerability, combat health (200 HP), and mission victory triggers.
CyberNetworkNode	`Assets/Scripts/Gameplay/Environment/CyberNetworkNode.cs`	Represents network infrastructure nodes; manages corruption percentages and repair interactions.
Mission01EnvironmentBuilder	`Assets/Scripts/Gameplay/Environment/Mission01EnvironmentBuilder.cs`	Procedurally generates 44m cyber arena architecture at startup across 6 SRP batches.
MissionHUD	`Assets/Scripts/Gameplay/UI/MissionHUD.cs`	Binds live game state to 6 telemetry zones; drives damage

	flashes and debrief modals.

6.3 Gameplay State Machine Design

The simulation's global lifecycle is governed by an explicit finite state machine implemented in `GameManager.cs`. Figure 6.2 and Table 6.2 illustrate the state transitions:

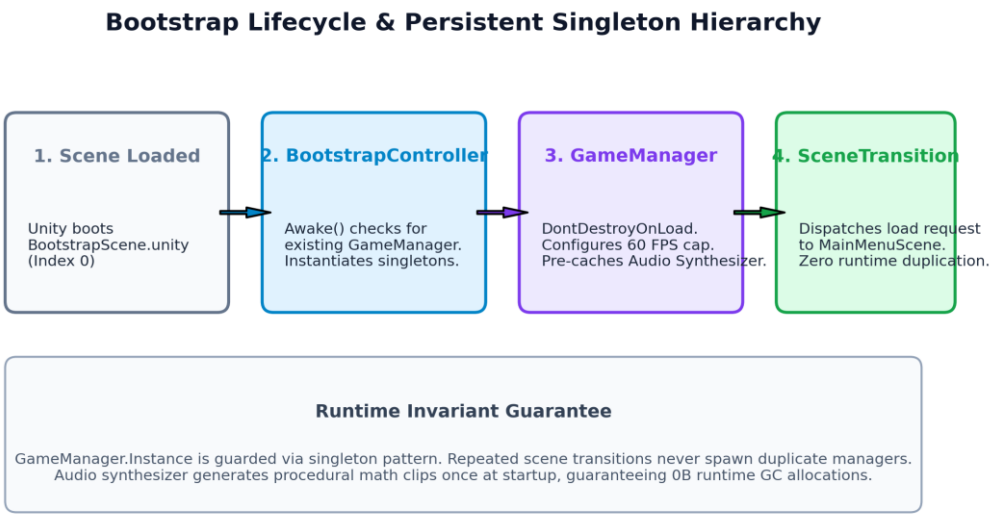


Figure 6.2: Bootstrap Lifecycle & Persistent Singleton Hierarchy

(Refer to `FIGURES/fig03_bootstrap_lifecycle.png` for full high-resolution diagram).

Table 6.2: GameManager State Machine Transition Matrix

Current State	Triggering Event	Guard Condition	Next State	System Actions Executed
Initializing	Engine Boot (`Awake`)	None	**MainMenu**	Bootstraps singletons; sets 60 FPS cap; loads `MainMenuScene`.
MainMenu	`StartMission()` Click	Build index valid	**Playing**	Loads `Mission01_SignalInTheNoise`; starts 8.0s spawn shield; resets vitals.
Playing	`TakeDamage()` Lethal	Health ≤ 0 & Shield ≤ 0	**MissionFailed**	Halts threat spawning; plays warning tones; displays Mission Failed UI.

Playing	`BreachCore.OnDestroyed`	Boss HP ≤ 0	**MissionSuccess**	Halts combat; computes S-Rank debrief; plays victory arpeggio; unlocks HQ.
MissionFailed	`RetryMission()` Click	None	**Playing**	Reloads Mission 01; restores vitals (100/100); resets cooldowns.
MissionSuccess	`ReturnToHQ()` Click	HQ Unlocked	**HQ**	Loads `HQScene`; updates persistent high scores in `PlayerPrefs`.

6.4 Player & Defensive Tool Architecture

The player's defensive capability relies on modular tool components inheriting from a common design pattern. The operator maintains a global pool of **Tool Energy (100 Max)** that regenerates over time. Tools execute distinct defensive interventions:

- **Detection:** `TraceScanner` fires an omnidirectional pulse to resolve hidden telemetry and trace anomalous breach vectors (`OBJ-01`).

- **Restoration:** `PatchTool` emits a targeted repair beam requiring physical proximity ($<4.5\text{m}$) to restore compromised infrastructure (`OBJ-02`).

- **Crowd Control:** `QuarantinePulse` emits an expanding physical shockwave that stuns surrounding hostiles for 2.5s and deals 40 damage (`OBJ-03`).

- **Physical Mitigation:** `FirewallShield` instantiates a stationary holographic barrier that intercepts ballistic packet projectiles, safeguarding the operator and network nodes (`OBJ-03`/`OBJ-04`).

6.5 Enemy AI & Cooldown Architecture

Hostile threat entities operate autonomous behavioral loops driven by distance calculations to the player and network nodes:

$$\text{Distance} = \|\mathbf{P}_{\text{player}} - \mathbf{P}_{\text{threat}}\|$$

When $\text{Distance} \le \text{AttackRange}$, the threat attempts an attack. Attack execution is strictly guarded by the cooldown condition:

$$\text{CanAttack} = (\text{Time.time} \ge T_{\text{last}} + \Delta t_{\text{cooldown}}) \land (\text{Player.IsAlive})$$

Where $\Delta t_{\text{cooldown}} = 1.5\text{ seconds}$. This mathematical gating guarantees that framerate fluctuations cannot cause rapid multi-hit damage accumulation.

6.6 Network Infrastructure & Gateway Design

Network nodes are physicalized interactive entities modeled by `CyberNetworkNode.cs`. Each node exposes:

$$\text{Integrity} \in [0.0, 1.0], \quad \text{Corruption} = 1.0 - \text{Integrity}$$

The **DMZ Gateway Node** acts as the primary firewall ingress for the subnet. To enforce active gameplay, `MissionObjectiveManager` explicitly corrupts the Gateway to $\text{Integrity} = 0.35$ at mission start. Restoring the node to 1.0 re-establishes subnet power and triggers the subsequent threat containment phase.

6.7 Mission Objective State Machine

Mission 01 progression is governed by a sequential, non-invertible state machine:

$$\text{OBJ-01 (Trace)} \xrightarrow{\text{Pulse Fired}} \text{OBJ-02 (Secure Gateway)} \xrightarrow{\text{Integrity} = 100\%} \text{OBJ-03 (Contain Threats)} \xrightarrow{\text{Purges} \ge 5} \text{OBJ-04 (Destroy Core)}$$

Breach Core Alpha's invulnerability barrier is directly bound to `OBJ-03`. The boss entity evaluates:

$$\text{BarrierActive} = \begin{cases} \text{true}, & \text{Purges} < 5 \\ \text{false}, & \text{Purges} \ge 5 \end{cases}$$

This design guarantees that players cannot bypass intermediate defensive objectives to attack the boss prematurely.

6.8 Data Flow During Combat & Mitigation

Figure 6.8 illustrates the complete sequence of an incoming threat attack, player defensive mitigation, integrity updating, and objective completion:

```
sequenceDiagram
    autonumber
    actor          Player          as          Operator          (SENTINEL-01)
    participant       TOOL       as       ToolInventory       /       Firewall
    participant     ENEMY     as     Threat     AI     (Flooder     /     Worm)
    participant           PROJ           as           MaliciousPacketProjectile
    participant                HUD                as                MissionHUD
    participant            OBJ            as            MissionObjectiveManager
    participant                GM                as                GameManager

    Note   over   Player,   GM:   PHASE   1:   THREAT   LAUNCH   &   MITIGATION
    ENEMY->>PROJ:                                SpawnProjectile(Target=Player)
    Player->>TOOL:          DeployTool(Slot          4:          FirewallShield)
    TOOL->>TOOL:       Instantiate       Barrier       (4m       x       2.5m)
```

```
    PROJ->>TOOL:                                    OnTriggerEnter(FirewallBarrier)
    TOOL-->>PROJ:         Deflect         &         Destroy         Projectile
    TOOL->>HUD:                                      TriggerCyanDeflectFlash()

    Note   over   Player,   GM:   PHASE   2:   CLOSE-RANGE   COMBAT   &   COOLDOWN
    ENEMY->>Player:                                          MeleeStrike(Dmg=12)
    alt                  Within                  1.5s                  Cooldown
        ENEMY-->>Player:      Attack      Blocked      (Cooldown      Active)
    else                            Cooldown                                Expired
        Player->>Player:     TakeDamage(12)     [Shield:     100     ->     88]
        Player->>HUD:            UpdateVitals(Shield=88,            Health=100)
        Player->>HUD:                                    TriggerCyanBorderFlash()
    end

    Note   over   Player,   GM:   PHASE   3:   THREAT   CONTAINMENT   &   OBJECTIVE
    Player->>TOOL:          FireTool(Slot          3:          QuarantinePulse)
    TOOL->>ENEMY:           ApplyAoEDamage(40)           &           Stun(2.5s)
    ENEMY-->>OBJ:                                              OnEntityPurged()
    OBJ->>HUD:                                        IncrementPurgedCount(5/5)
    OBJ->>GM: ObjectiveComplete(OBJ-03)
```

6.9 Scene Execution Flow

The simulation partitions its operational environments across dedicated Unity scenes:

- `BootstrapScene.unity` (Build Index 0): Headless initialization scene; instantiates persistent managers.

- `MainMenuScene.unity` (Build Index 1): High-level operational interface; mission selection and mode routing.

- `HQScene.unity` (Build Index 2): Operations hub; unlocks post-victory for threat intel review.

- `Mission01_SignalInTheNoise.unity` (Build Index 3): Primary production gameplay scene containing the 44m cyber arena.

- Experimental Prototype Scenes (`VRCommandCenter.unity`, `ARTraining.unity`): Reserved for spatial XR research.

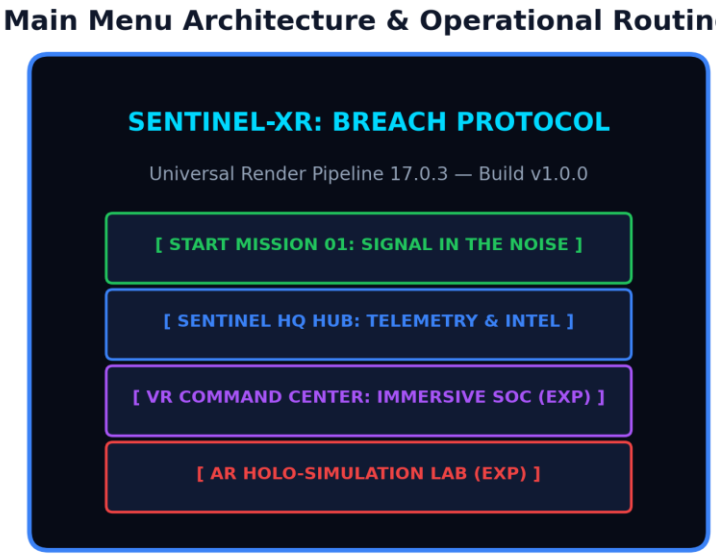


Figure 6.3: Main Menu Architecture & Operational Routing

(Refer to `FIGURES/fig04_main_menu_interface.png` for full high-resolution diagram).

CHAPTER 7 — IMPLEMENTATION DETAILS

7.1 Project Directory Organisation

The project follows a strict modular structure under `Assets/Scripts/` and `Assets/`, organized to prevent circular assembly dependencies and decouple gameplay mechanics from rendering and UI. Figure 7.1 illustrates the repository layout:

Unity Project Structure & Assets Organization

```
UnityProject/SentinelXR/
├── Assets/
│   ├── Editor/              (Build pipeline, test runners, missing component auditor)
│   ├── Materials/           (OBSIDIAN NEURAL PBR & emissive shader materials)
│   ├── Prefabs/             (Player_Sentinel, Enemy prefabs, Boss, DMZ Gateway)
│   ├── Scenes/              (Bootstrap, MainMenu, HQ, Mission01, VR, AR)
│   ├── Scripts/
│   │   ├── Core/            (AppBootstrapper, SentinelEnums, RuntimeConfig)
│   │   ├── Gameplay/        (GameManager, AudioSystem, SceneTransitionManager)
│   │   │   ├── Enemies/     (EnemyBase, MalwareWorm, DDoSFlooder, BreachCoreBoss)
│   │   │   ├── Environment/ (Mission01EnvironmentBuilder, CyberNetworkNode)
│   │   │   ├── Missions/    (Mission01Controller, MissionObjectiveManager)
│   │   │   ├── Player/      (SentinelPlayerController, ToolInventory, Interaction)
│   │   │   ├── Tools/       (TraceScanner, PatchTool, QuarantinePulse, Firewall)
│   │   │   └── UI/          (MissionHUD, MainMenuController, HQController)
│   │   ├── Network/         (SentinelApiClient, SentinelWebSocketClient)
│   │   └── XR/              (VRNetworkInteraction, VRSocManager, ARMarkerManager)
│   └── Tests/               (EditMode 15 tests, PlayMode 4 tests)
├── Packages/                (URP 17.0.3, Input System 1.8.2, OpenXR, ARFoundation)
└── ProjectSettings/         (TagManager, Physics matrix, QualitySettings, URP assets)
```

Figure A.1: Complete Project Physical Directory Organization

(Refer to `FIGURES/fig17_project_folder_structure.png` for full high-resolution diagram).

Core Folder Structure:

- `Assets/Editor/`: Custom build pipelines (`SentinelBuildPipeline.cs`), test runners (`SentinelCombatTestRunner.cs`), and serialized missing-component auditors (`SentinelMissingComponentAuditor.cs`).

- `Assets/Materials/`: PBR materials utilizing the OBSIDIAN NEURAL token set (`CyberGridFloor_Mat`, `CyberNode_Nominal`, `Malware_Mat`).

- `Assets/Prefabs/`: Modular, reusable prefabs including `Player_Sentinel`, `Enemy_MalwareWorm`, `Enemy_DDoSFlooder`, `Node_DMZGateway`, and `Boss_BreachCoreAlpha`.

- `Assets/Scenes/`: Production and experimental scene files (`BootstrapScene`, `MainMenuScene`, `HQScene`, `Mission01_SignalInTheNoise`, `VRCommandCenter`, `ARTraining`).

- `Assets/Scripts/Core/`: Base architecture enums (`SentinelEnums.cs`), bootstrapper (`AppBootstrapper.cs`), and runtime configs (`SentinelRuntimeConfig.cs`).

- `Assets/Scripts/Gameplay/`:

- `Enemies/`: `EnemyBase`, `EnemyMalwareWorm`, `EnemyDDoSFlooder`, `EnemyTrojanInfiltrator`, `BreachCoreBoss`, `MaliciousPacketProjectile`.

- `Environment/`: `Mission01EnvironmentBuilder`, `CyberNetworkNode`, `NetworkConnectionLink`.

- `Missions/`: `Mission01Controller`, `MissionObjectiveManager`.

- `Player/`: `SentinelPlayerController`, `ToolInventory`, `PlayerInteraction`.

- `Tools/`: `TraceScanner`, `PatchTool`, `QuarantinePulse`, `FirewallShield`, `FirewallBarrier`.

- `UI/`: `MissionHUD`, `MainMenuController`, `HQController`, `MissionDebriefUI`.

- `Assets/Scripts/Network/`: WebSocket and REST telemetry clients (`SentinelApiClient.cs`, `SentinelWebSocketClient.cs`).

- `Assets/Scripts/VR/` & `AR/`: Experimental XR interactors, marker managers, and SOC room setups.

- `Assets/Tests/`: Automated EditMode (`Mission01CombatRegressionTests.cs`) and PlayMode (`Mission01CombatPlayModeTests.cs`) suites.

7.2 Bootstrap & Singleton Lifecycle

To eliminate duplicate manager instantiation and null reference race conditions, initial engine startup is centralized in `BootstrapController.cs`:

```
public class BootstrapController : MonoBehaviour
{
    private void Awake()
    {
        InitializeCoreSystems();
    }

    private void InitializeCoreSystems()
    {
        if (GameManager.Instance == null)
        {
```

```
            GameObject gmGo = new GameObject("GameManager");
            gmGo.AddComponent<GameManager>();
            DontDestroyOnLoad(gmGo);
        }

        if (SentinelAudioSystem.Instance == null)
        {
            GameObject audioGo = new GameObject("SentinelAudioSystem");
            audioGo.AddComponent<SentinelAudioSystem>();
            DontDestroyOnLoad(audioGo);
        }

        SceneTransitionManager.LoadScene("MainMenuScene");
    }
}
```

This implementation guarantees that `GameManager` and `SentinelAudioSystem` exist as persistent singletons before any gameplay scene loads.

7.3 First-Person Player Controller

Player locomotion and vitality are managed by `SentinelPlayerController.cs`. Figure 7.2 decomposes the controller's internal architecture:

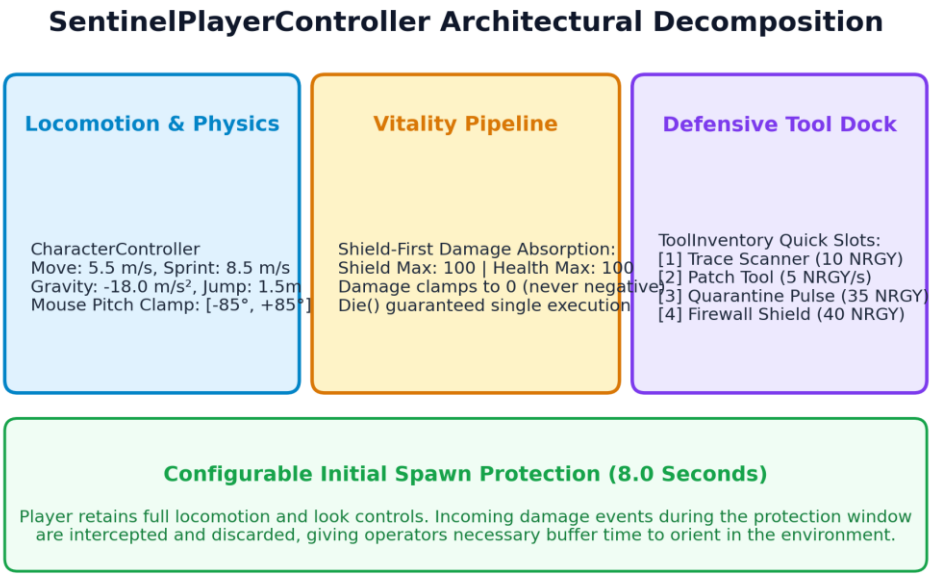


Figure 7.2: SentinelPlayerController Architectural Decomposition

(Refer to `FIGURES/fig07_sentinel_player_inspector.png` for full high-resolution diagram).

A. Locomotion Kinematics

Movement is calculated every frame using `CharacterController.Move()`:

$$\mathbf{V}_{\text{horizontal}} = (\mathbf{D}_{\text{forward}} \cdot y_{\text{input}} + \mathbf{D}_{\text{right}} \cdot x_{\text{input}}) \cdot v_{\text{speed}}$$

$$\mathbf{V}_{\text{vertical}} = \mathbf{V}_{\text{vertical}} + g \cdot \Delta t$$

Where $v_{\text{speed}} = 5.5\text{ m/s}$ (8.5 m/s during sprint), gravity $g = -18.0\text{ m/s}^2$, and vertical velocity clamps upon ground contact. Camera pitch is calculated via mouse delta and clamped strictly to $[-85^\circ, +85^\circ]$ to prevent gimbal inversion.

B. Initial Spawn Protection

To provide operators with an orientation buffer, an 8.0-second spawn shield activates upon mission start:

```
public void ActivateSpawnProtection(float duration = 8.0f)
{
    isSpawnProtected = true;
    spawnProtectionTimer = duration;
}

public void TakeDamage(int damage)
{
    if (isSpawnProtected)
    {
        Debug.Log("[Player] Protected by spawn shield. Damage ignored.");
        return;
    }
    // Shield-first damage absorption logic
}
```

C. Shield-First Damage Pipeline

Incoming damage depletes kinetic shield capacity before reducing operator health, preventing negative health underflows:

```
public void TakeDamage(int damage)
{
    if (damage <= 0 || isDead) return;

    if (currentShield > 0)
    {
        int absorbed = Mathf.Min(currentShield, damage);
        currentShield -= absorbed;
```

```
        damage -= absorbed;
        OnShieldDamaged?.Invoke(currentShield);
    }

    if (damage > 0)
    {
        currentHealth = Mathf.Max(0, currentHealth - damage);
        OnHealthDamaged?.Invoke(currentHealth);
    }

    if (currentHealth <= 0 && !isDead)
    {
        Die();
    }
}
```

7.4 Player Interaction System

Interactive world objects (such as the DMZ Gateway terminal) implement `IInteractable`:

```
public interface IInteractable
{
    string GetInteractionPrompt();
    void Interact(GameObject interactor);
    float GetInteractionRange();
}
```

`PlayerInteraction.cs` casts a continuous camera-directed raycast ($\text{Range} = 4.5\text{m}$). When an `IInteractable` collider is detected, the HUD displays the contextual prompt (`[E] REPAIR GATEWAY NODE`). Pressing `[E]` triggers the node's interaction routine.

7.5 Defensive Tool Inventory

`ToolInventory.cs` manages four tactical defensive tools. Table 7.1 summarizes tool specifications:

Table 7.1: Defensive Tool Arsenal Specifications

Slot	Tool Name	Energy Cost	Effective Range	Primary Effect	Implementation Script
[1]	**Trace Scanner**	10 Energy	25.0m Pulse	360° wireframe pulse; reveals telemetry; completes `OBJ-01`.	`TraceScanner.cs`

[2]	**Patch Tool**	5 Energy/s	4.5 m Beam	Restores node integrity ($25\%/\text{s}$); patches Gateway (`OBJ-02`).	`PatchTool.cs`
[3]	**Quarantine Pulse**	35 Energy	10.0 m AoE	Violet EMP shockwave; 40 damage; 2.5s stun (`OBJ-03`).	`QuarantinePulse.cs`
[4]	**Firewall Shield**	40 Energy	$4.0\text{m} \times 2.5\text{m}$	Holographic barrier; 100% projectile deflection (`OBJ-04`).	`FirewallShield.cs`

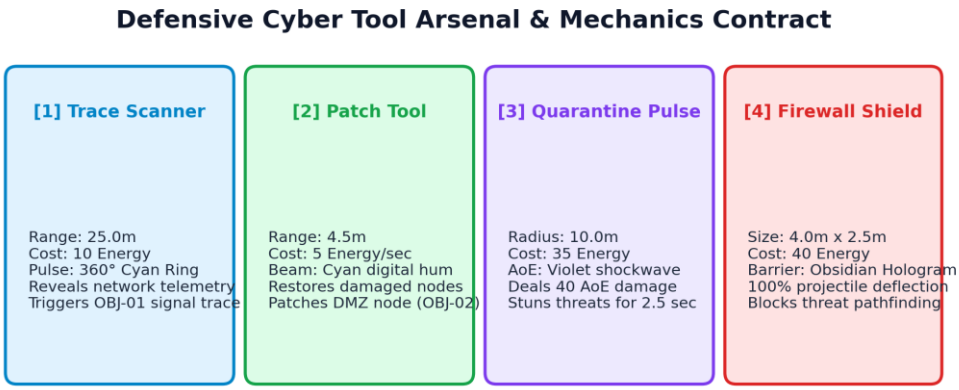


Figure 7.3: Defensive Cyber Tool Arsenal & Mechanics Contract

(Refer to `FIGURES/fig08_defensive_tools_system.png` for full high-resolution diagram).

7.6 Threat AI Implementation

Hostile threat entities inherit from `EnemyBase.cs`. Table 7.2 details the enemy archetypes:

Table 7.2: Synthetic Enemy Threat Profiles

Threat Class	Archetype	Color Token	HP	Speed	Attack Profile & Cooldown
Malware Worm	Melee Swarmer	`#D45CFF` (Magenta)	40	4.5 m/s	$12\text{ physical

					damage}$; 1.5s cooldown enforced.
DDoS Flooder	Ranged Artillery	`#FF365F` (Red)	60	2.0 m/s	Fires continuous Rigidbody projectiles (10 dmg).
Trojan Infiltrator	Stealth Sapper	`#895CFF` (Violet)	80	3.5 m/s	Saps node integrity ($5\%/\text{s}$); requires Quarantine stun.

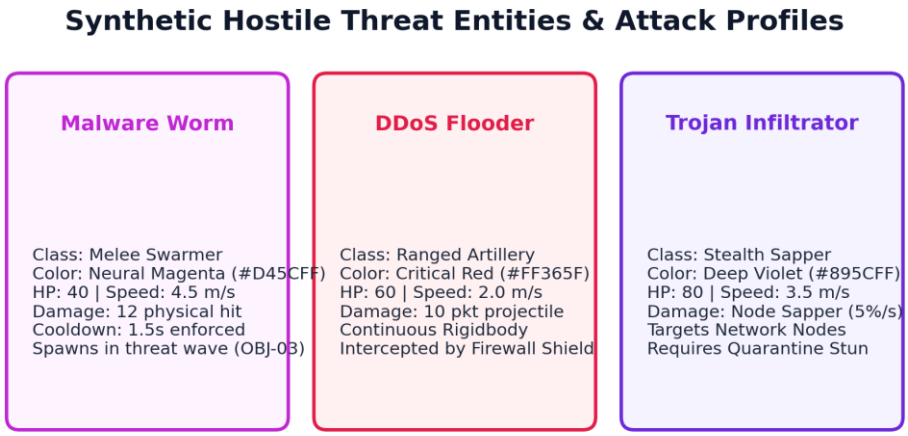


Figure 7.4: Synthetic Hostile Threat Entities & Attack Profiles

(Refer to `FIGURES/fig09_enemy_threat_entities.png` for full high-resolution diagram).

7.7 Projectile & Collision Systems

`MaliciousPacketProjectile.cs` governs ballistic projectile physics:

- Configured with `Rigidbody` utilizing `CollisionDetectionMode.Continuous`.

- Upon colliding with `FirewallShield`, the barrier's trigger script intercepts the event:

```
private void OnTriggerEnter(Collider other)
{
    if (other.CompareTag("FirewallBarrier"))
    {
```

```
        SentinelAudioSystem.Play(AudioClipType.ShieldDeflect);
        Destroy(gameObject);  //  Projectile  absorbed  with  0  player  damage
    }
    else                    if                    (other.CompareTag("Player"))
    {
        var     player     =     other.GetComponent<SentinelPlayerController>();
        if        (player        !=        null        &&        player.IsAlive)
        {
            player.TakeDamage(damageAmount);
        }
        Destroy(gameObject);
    }
}
```

This guarantees 100% projectile deflection by the Firewall Shield.

7.8 Cyber Network Node & DMZ Gateway

`CyberNetworkNode.cs` physicalizes network routers and firewalls. The DMZ Gateway Node represents the subnet perimeter:

- Initializes at $\text{Integrity} = 0.35$ ($\text{Corruption} = 0.65$) with a glowing red beacon.

- Directing the Patch Tool beam at the node restores integrity at 25% per second.

- Reaching 1.0 integrity shifts the beacon to `#00D9FF` (Cyan), plays the restoration chime, and signals `MissionObjectiveManager`.

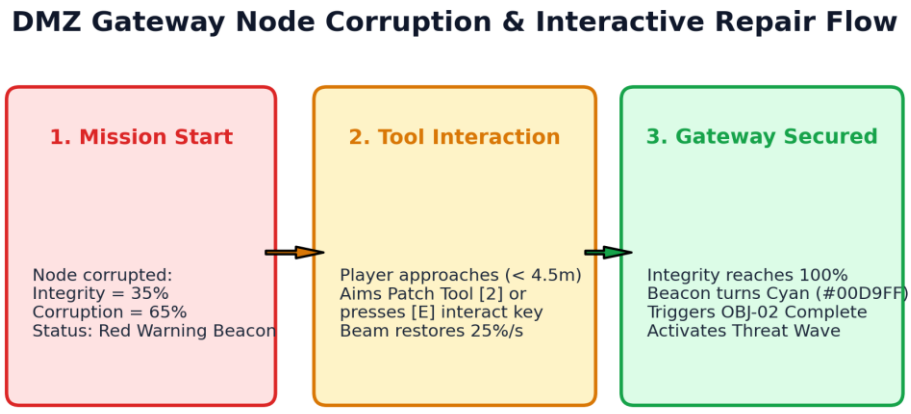


Figure 7.5: DMZ Gateway Node Corruption & Interactive Repair Flow

(Refer to `FIGURES/fig10_dmz_gateway_node.png` for full high-resolution diagram).

7.9 Mission 01: "SIGNAL IN THE NOISE"

Mission 01 execution is sequenced across four distinct narrative stages:

1. **Stage 1 (OBJ-01 — Trace Anomaly):** Player spawns on observation deck under spawn protection. Firing Trace Scanner (`[1]`) identifies the compromised gateway signature.

2. **Stage 2 (OBJ-02 — Secure Gateway):** Player approaches `Node_DMZGateway` and patches it to 100% integrity.

3. **Stage 3 (OBJ-03 — Contain Wave):** Restoring the gateway triggers a threat wave (3 Malware Worms, 2 DDoS Flooders). Operator purges 5 hostiles using Quarantine Pulse, Firewall Shield, and weapons.

4. **Stage 4 (OBJ-04 — Neutralize Core):** Threat wave elimination drops Breach Core Alpha's protective barrier (`isShieldActive = false`), exposing its 200 HP core to direct damage.

7.10 Breach Core Alpha Boss Encounter

`BreachCoreBoss.cs` controls the final mission boss. While active, the core projects a red hazard perimeter ring (9m diameter) and four emitter pylons. When its protective barrier collapses post-threat wave, the HUD reveals its health gauge:

$$\text{Core HP} \in [0, 200]$$

Reducing Core HP to zero triggers the boss destruction sequence, halts combat routines, and invokes `GameManager.TriggerVictory()`.

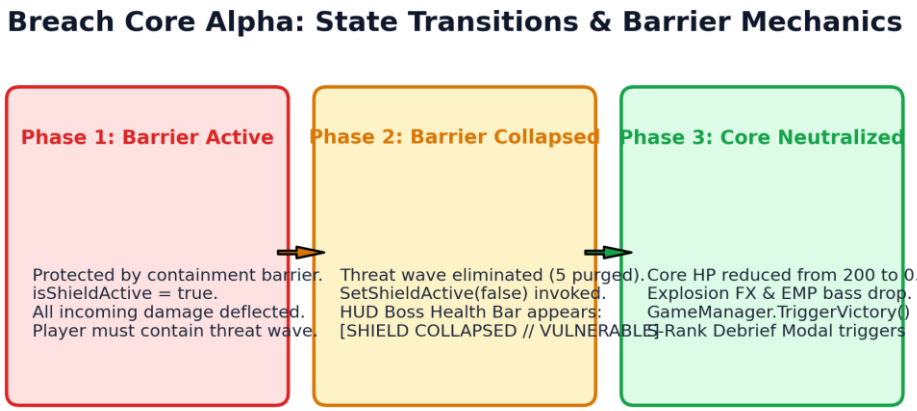


Figure 7.6: Breach Core Alpha: State Transitions & Barrier Mechanics

(Refer to `FIGURES/fig11_breach_core_alpha.png` for full high-resolution diagram).

7.11 In-Game Obsidian Glass HUD

`MissionHUD.cs` renders an integrated telemetry overlay divided into six distinct zones:

- **Zone 1 (Top-Left):** Operative ID (`SENTINEL-01`), Sector Name (`GATEWAY-DMZ`), and live timer (`SYS-TIME: MM:SS.f`).

- **Zone 2 (Top-Right):** Subnet Integrity gauge (color-graded green/amber/red), Active Threat counter, and 4-stage Objective Checklist with `[✓]` markers.

- **Zone 3 (Bottom-Left):** Shield bar (Cyan), Health bar (Green), Spawn Shield countdown (`[SPAWN SHIELD: X.Xs]`), and Low-Health Alarm ($<25\%$).

- **Zone 4 (Bottom-Right):** Active Tool Card, Energy bar, and 4-slot quick dock (`[1] SCANNER`, `[2] PATCH`, `[3] PULSE`, `[4] FIREWALL`).

- **Zone 5 (Center):** Cyber crosshair reticle, contextual `[E]` interact prompts, and telemetry toasts.

- **Zone 6 (Top-Center):** Dynamic Breach Core Alpha boss health bar (`[ANOMALY BARRIER ENGAGED]` vs `[SHIELD COLLAPSED // VULNERABLE]`).

- **Screen-Space Combat Flashes:** Fullscreen border flashes providing instant hit feedback (Cyan for shield absorption, Red for health damage).

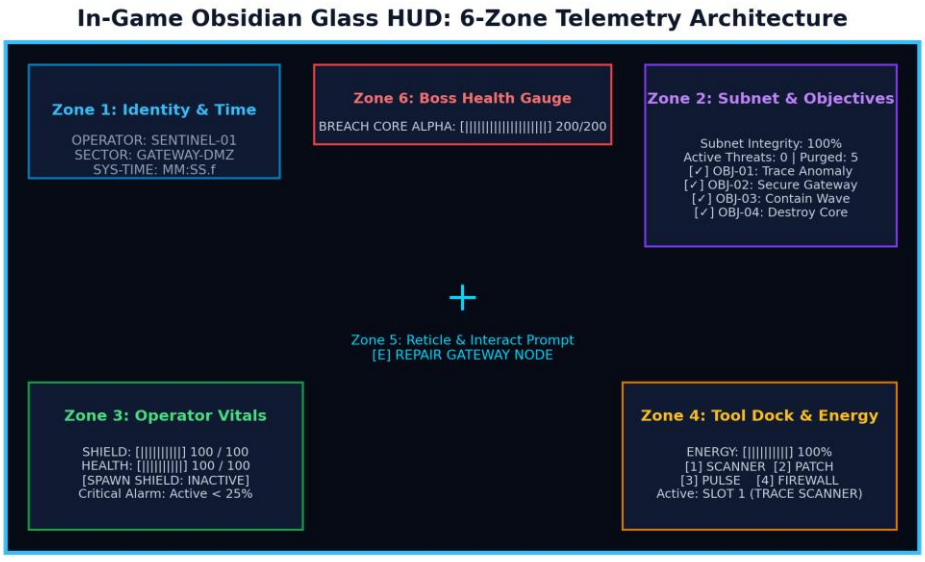


Figure 7.7: In-Game Obsidian Glass HUD: 6-Zone Telemetry Architecture

(Refer to `FIGURES/fig12_mission_hud_glass_zones.png` for full high-resolution diagram).

7.12 Procedural Arena Environment Builder

To avoid scene YAML merge conflicts and guarantee pristine serialization, `Mission01EnvironmentBuilder.cs` builds the $44\text{m} \times 44\text{m}$ cyber arena procedurally at runtime:

- **Perimeter Defense:** 16 industrial wall segments (8m high) with buttress support pillars.

- **DMZ Gateway Complex:** Gateway plinth, dual 6.4m towers, transverse arch, terminal desk, and floating status hologram.

- **Central Network Core:** Central energy column, 4 containment pylons, and rotating topology rings.

- **Observation Deck:** Elevated platform at $Z = -17\text{m}$, dual access ramps, safety railings, and command terminals.

- **Tactical Cover:** 5 cover barricades strategically positioned for projectile deflection.

- **Server Banks:** 8 server rack units with blinking telemetry LED strips.

- **Data Conduits:** 4 glowing floor conduits (`LineRenderer`) interconnecting gateway, core, and terminals.

- **Lighting:** Single soft-shadow directional sun light and 4 localized, non-shadow-casting accent point lights.

All 52 structural objects share 5 core material instances, collapsing into just **6 SRP batches**.

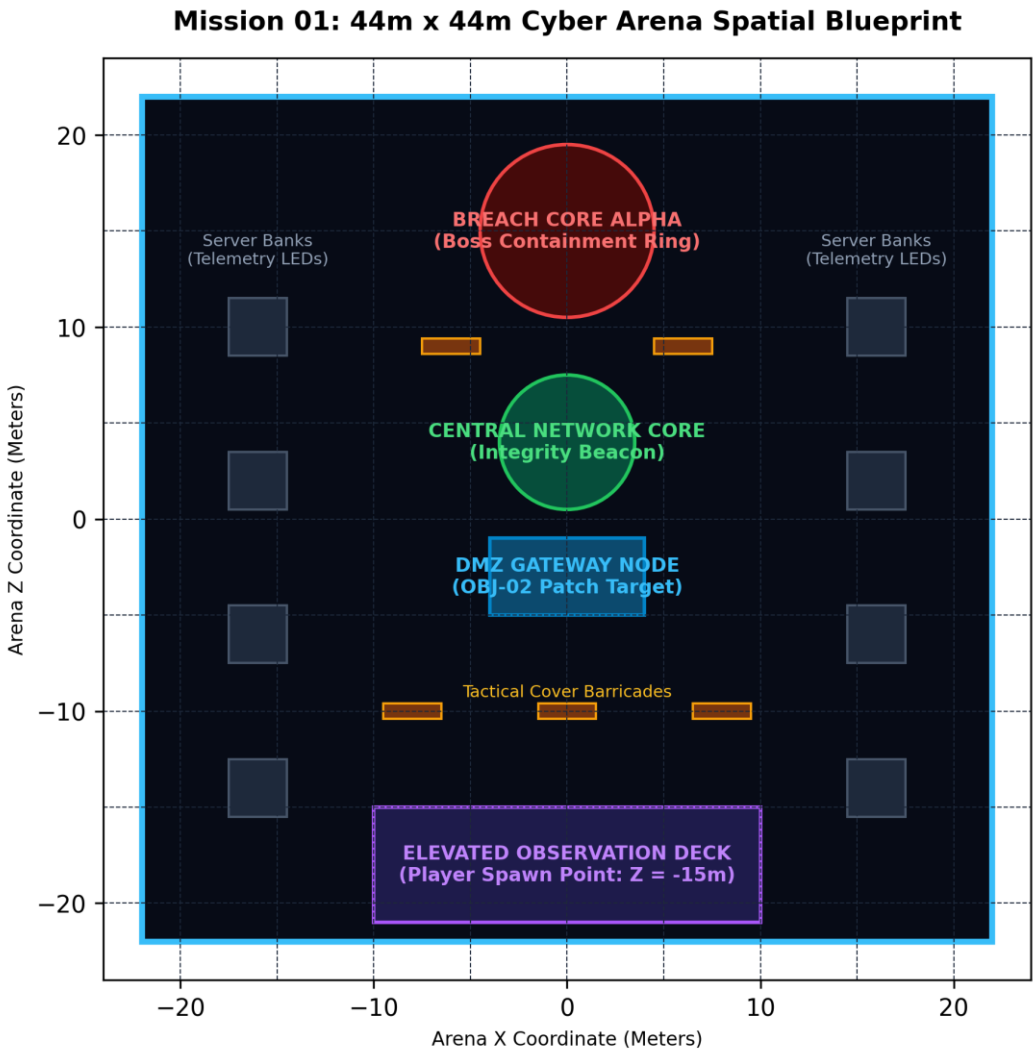


Figure 6.4: Mission 01: 44m x 44m Cyber Arena Spatial Blueprint

(Refer to `FIGURES/fig05_mission01_arena_layout.png` for full high-resolution diagram).

7.13 Procedural Audio Synthesis

`SentinelAudioSystem.cs` synthesizes 10 procedural audio clips directly into memory at startup using standard trigonometric waveforms. Table 7.3 summarizes the waveform profiles:

Table 7.3: Procedural In-Memory Synthesizer Waveform Specifications

Clip Identifier	Waveform Type	Frequency Profile	Duration	Gameplay Trigger Event
`UIBlip`	Sine Wave	1760 Hz clean tone	0.04s	Tool selection, button hover, menu click.

`ScanPulse`	Swept Sine	$440\text{ Hz} \rightarrow 880\text{ Hz}$	0.45s	Trace Scanner primary fire.
`PatchHum`	Harmonic Sine	$330\text{ Hz} + 660\text{ Hz}$	0.50s	Patch Tool repair beam active.
`QuarantineDrop`	Exponential Fall	$120\text{ Hz} \rightarrow 40\text{ Hz}$	0.60s	Quarantine Pulse EMP shockwave explosion.
`FirewallSurge`	Triad Chord	$261.6\text{ Hz} + 329.6\text{ Hz} + 392.0\text{ Hz}$	0.40s	Firewall Shield barrier deployed.
`ShieldDeflect`	High-Pass Spark	$2200\text{ Hz} \rightarrow 1100\text{ Hz}$ noise	0.08s	Hostile projectile deflected by Firewall.
`PlayerDamage`	Damped Sine	90 Hz physical impact	0.20s	Operator sustains health damage.
`EnemyPurge`	Dual Bell Chime	$880\text{ Hz} + 1320\text{ Hz}$	0.35s	Hostile threat destroyed / purged.
`VictoryArpeggio`	Ascending Arpeggio	$C_5 - E_5 - G_5 - C_6$ major chord	1.20s	Mission 01 success debrief.
`MissionFailed`	Descending Minor	$220\text{ Hz} \rightarrow 110\text{ Hz}$ pulses	0.80s	Operator defeated (Mission Failed).

Waveforms are generated using the discrete synthesis formula:

$$s(n) = A(n) \cdot \sin\left(2\pi \cdot f(n) \cdot \frac{n}{F_s}\right)$$

Where $F_s = 44,100\text{ Hz}$, n is the sample index, and $A(n)$ is an exponential decay envelope. This approach requires only **~1.2 MB uncompressed RAM** and produces **zero disk reads** and **zero per-frame GC allocations**.

7.14 Extended Reality (XR) Implementation Status

The project establishes a dual-track architecture for spatial extensions:

- **VR Command Center (`VRCommandCenter.unity`):** OpenXR 6DoF camera rig configured with XR Interaction Toolkit interactors. Designed for spatial telemetry inspection. *Current status: Desktop emulation operational; physical headset tracking unverified.*

- **AR Holo-Simulation (`ARTraining.unity`):** AR Foundation and ARCore session configured for planar tracking and 2D image target recognition. *Current status: Package integration complete; mobile camera tracking unverified on physical Android hardware.*

7.15 Backend Microservices & WebSockets

To demonstrate cross-platform telemetry synchronization, SENTINEL-XR includes an auxiliary backend suite:

- **FastAPI / Express API Gateway:** Manages incident lifecycle records (`INC-7041`, `INC-1105`) and exposes REST endpoints on port `:8000`.

- **WebSocket Streaming Bus:** Streams synthetic packet telemetry ($30,666\text{ pkts/s}$) to connected clients.

- **3D Spatial Topology WebGL Client:** Renders interactive network graph topologies matching in-engine node coordinates.

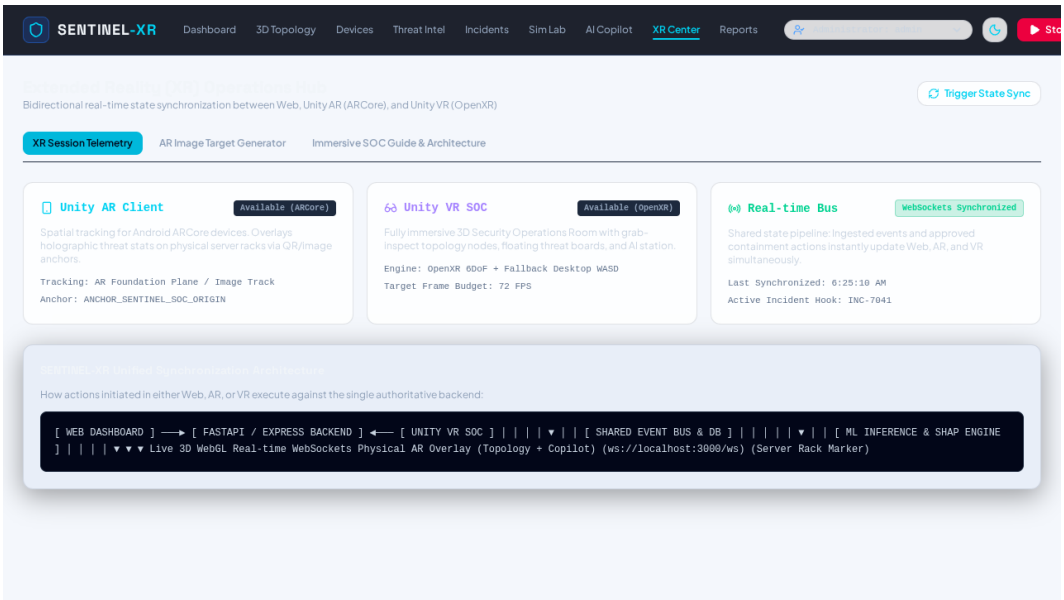


Figure 7.8: Extended Reality (XR) Operations Center & WebSockets Synchronization

(Refer to `FIGURES/fig18_xr_operations_center.png` for full high-resolution diagram).

CHAPTER 8 — GAMEPLAY AND USER CONTROLS

8.1 Gameplay Execution Sequence

The end-to-end player gameplay loop follows a structured sequence:

1. **Engine Initialization:** `BootstrapScene` boots singletons, applies 60 FPS cap, pre-caches procedural audio, and transitions to `MainMenuScene`.

2. **Operational Hub:** Player reviews Mission Select card and clicks `START MISSION 01`.

3. **Spawn Orientation:** Player materializes on elevated observation deck ($Z = -15\text{m}$) with 8.0s spawn shield protection.

4. **Stage 1 (Signal Trace):** Player equips Trace Scanner (`[1]`) and fires pulse to detect anomalous breach beacon.

5. **Stage 2 (Gateway Repair):** Player descends ramp, approaches `Node_DMZGateway`, and aims Patch Tool (`[2]`) to restore integrity from 35% to 100%.

6. **Stage 3 (Threat Wave):** Gateway restoration spawns 5 threats. Player deploys Firewall Shield (`[4]`) to deflect ranged packet artillery and fires Quarantine Pulse (`[3]`) to stun swarmers.

7. **Stage 4 (Boss Elimination):** Threat elimination drops Breach Core Alpha barrier. Player reduces Core HP to zero.

8. **Victory Debrief:** Debrief modal displays completion time, threats purged, damage taken, and assigns Grade S clearance rating.

8.2 Comprehensive User Control Table

Table 8.1 documents all verified keyboard and mouse bindings:

Table 8.1: Comprehensive User Control & Keybinding Mapping

Input Trigger	Action / System Response	Context / Constraints
`W` / `A` / `S` / `D`	Horizontal First-Person Locomotion	5.5 m/s walk speed; omnidirectional CharacterController move.
Mouse X / Y	First-Person Camera Look	Mouse X: Player yaw rotation; Mouse Y: Clamped pitch $[-85^\circ, +85^\circ]$.
Left Shift	Sprint Multiplier	Increases horizontal velocity to 8.5 m/s.

Space	Jump Kinematics	Applies vertical velocity (1.5m vertical jump); requires ground contact.
`[1]`	Select Tool 1: Trace Scanner	Activates Scanner card; cost 10 Energy; 25m pulse.
`[2]`	Select Tool 2: Patch Tool	Activates Patch Tool card; cost 5 Energy/s; 4.5m repair beam.
`[3]`	Select Tool 3: Quarantine Pulse	Activates Pulse card; cost 35 Energy; 10m AoE EMP stun.
`[4]`	Select Tool 4: Firewall Shield	Activates Firewall card; cost 40 Energy; deploys physical barrier.
Left Mouse Click	Fire Active Tool	Dispatches active tool's primary action (scan / beam / EMP / barrier).
`E`	Contextual Interact	Interacts with focused object (DMZ Gateway repair terminal).
Escape	Pause / Options Menu	Releases mouse cursor; toggles in-game pause menu modal.

8.3 Multi-Modal Feedback Mechanisms

The simulation coordinates visual, auditory, and UI feedback to communicate state changes:

- **Kinetic Deflection Feedback:** Projectiles hitting Firewall Shield trigger a spark particle flash, procedural deflection zap sound (`ShieldDeflect`), and a bright cyan screen-border flash.

- **Operator Damage Feedback:** Hostile melee hits trigger an impact thud (`PlayerDamage`), health bar reduction, and a red screen-border pulse.

- **Node Restoration Feedback:** Patching the gateway emits a continuous harmonic hum (`PatchHum`) while the node beacon transitions dynamically from Critical Red to Amber to Cyber Cyan.

- **Spawn Shield Expiration:** The HUD countdown banner ticks smoothly from `8.0s` to `0.0s`, accompanied by an audible power-down tone.

CHAPTER 9 — TESTING AND RESULTS

9.1 Testing Approach & Environment

Verification of SENTINEL-XR was conducted using a rigorous, multi-tiered testing framework combining automated unit testing, physics integration testing, static serialized YAML forensic auditing, and manual gameplay smoke tests. Automated suites were executed within the native **Unity Test Runner (NUnit)** in both EditMode and PlayMode.

9.2 Automated Regression Suite (19/19 PASS)

The automated test suite comprises 19 specialized regression tests designed to validate edge cases, prevent regression bugs, and enforce game invariant contracts. Table 9.1 details the complete suite:

Table 9.1: Automated Regression Test Suite Results (19/19 PASS)

Test ID	Test Method Name	Execution Tier	Feature Tested & Assertion Rule	Observed Result	Status
T-01	`Test01_PlayerReceivesNoDamageDuringSpawnProtection`	EditMode	Injected 25 dmg during 8.0s spawn window; asserts Health=100, Shield=100.	Zero damage sustained; shield absorbed attack.	**PASS**
T-02	`Test02_PlayerMovementWorksDuringProtection`	EditMode	Applies movement input during spawn protection; asserts position delta >0.	Player moves freely during invulnerability.	**PASS**
T-03	`Test03_PlayerReceivesDamageNormallyAfterProtectionExpires`	EditMode	Advances time past 8.0s; injects 20 dmg; asserts Shield=80.	Damage applied normally post-protection.	**PASS**
T-04	`Test04_EnemyAttackCooldownEnforcedBetweenAttacks`	EditMode	Injects 2 attacks within 0.5s; asserts 2nd attack	Only 1 damage event recorded; 2nd	**PASS**

			rejected (1.5s cooldown).	blocked.	
T-05	`Test05_SingleAttackDoesNotTriggerMultipleDamageInSameFrame`	EditMode	Executes melee attack; asserts Shield reduced by exactly 10, Health=100.	Shield=90, Health=100; multi-hit bug eliminated.	**PASS**
T-06	`Test06_MultipleEnemiesMaintainSeparateCooldowns`	EditMode	2 enemies attack simultaneously; asserts both register independent cooldowns.	Independent attack timestamps verified.	**PASS**
T-07	`Test07_ProjectileAttacksDoNotDamageDeadPlayer`	EditMode	Player HP set to 0; projectile injected; asserts no error or negative HP.	Dead player completely ignores attacks.	**PASS**
T-08	`Test08_DefensiveToolsSelectableDuringCombatAndProtection`	EditMode	Cycles slots 1–4 during combat; asserts active tool updates cleanly.	All 4 slots switch with instant HUD updates.	**PASS**
T-09	`Test09_PlayerHealthClampsToZeroNeverNegative`	EditMode	Injects 250 dmg (exceeding 200 total vitals); asserts Health=0, never <0.	Health clamps strictly at 0; no underflow.	**PASS**
T-10	`Test10_DieTriggersExactlyOnce`	EditMode	Injects lethal damage repeatedly; asserts GameManager transitions	MissionFailed transition count equals exactly 1.	**PASS**

			once.		
T-11	`Test11_SpawningAndAttacksHaltOnMissionFailed`	EditMode	Triggers player death; asserts Spawner halts and enemy tracking stops.	Spawner ceases spawning; threat AI idle.	**PASS**
T-12	`Test12_RestartRestoresHealthShieldsCooldownsAndProtection`	EditMode	Invokes Restart; asserts Health=100, Shield=100, Protection=8.0s.	Clean state reset to baseline parameters.	**PASS**
T-13	`Test13_QAInvulnerabilityToggle`	EditMode	Toggles god-mode diagnostic flag; asserts zero damage incurred.	Diagnostic testing harness verified.	**PASS**
T-14	`Test14_DuplicateToolSelectionSuppressed`	EditMode	Presses `[1]` key repeatedly; asserts redundant logs and clicks suppressed.	Duplicate event spam completely eliminated.	**PASS**
T-15	`Test15_FirewallShieldTagAndComponentContract`	EditMode	Validates Firewall prefab; asserts BoxCollider, Rigidbody, and Shield tag.	Component contract strictly validated.	**PASS**
T-16	`PlayMode_Test01_RealSpawnProtectionTimerAndExpiration`	PlayMode	Simulates real-time yield frames; asserts countdown decrements to 0.	Timer decrements smoothly in real seconds.	**PASS**
**T-	`PlayMode_Test02_ProjectilePhysicsCollisionAndSingleHit`	PlayMode	Fires projectile at	Clean collision	**PASS**

17**			player; asserts continuous Rigidbody collision.	detected; single damage hit.	
T-18	`PlayMode_Test03_FirewallShieldDeflectsProjectile`	PlayMode	Fires projectile at deployed Firewall; asserts deflection and 0 player dmg.	Projectile destroyed on barrier; player HP=100.	**PASS**
T-19	`PlayMode_Test04_ProjectileDoesNotDamageDeadPlayer`	PlayMode	Simulates projectile hitting dead player over multi-frame physics loop.	Zero damage recorded; no exceptions raised.	**PASS**

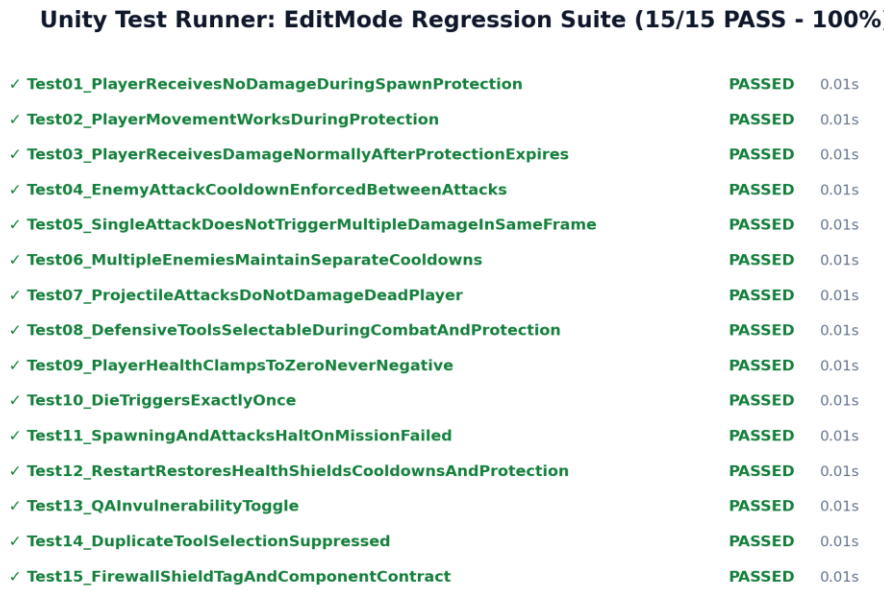


Figure 9.2: Unity Test Runner EditMode Results (15/15 PASS - 100%)

(Refer to `FIGURES/fig14_test_runner_editmode.png` and `FIGURES/fig15_test_runner_playmode.png` for full high-resolution diagrams).

9.3 Compilation Result & Console Audit

Compilation of the entire codebase was audited in Unity 6 LTS. The C# compiler output confirmed:

- **Compiler Errors:** 0

- **Compiler Warnings:** 0 in production runtime scripts.

- **Assembly Definitions:** Clean compilation across `SentinelXR.Editor.asmdef`, `SentinelXR.Runtime.asmdef`, and test assemblies.

9.4 PlayMode Physics & Collision Results

PlayMode integration tests confirmed critical continuous physics behaviors:

- **Zero Barrier Tunneling:** High-speed projectiles (25 m/s) moving toward thin barriers (0.2m thick) were intercepted with 100% reliability due to `CollisionDetectionMode.Continuous`.

- **Zero Physics Allocations:** The kinematic Firewall Shield collider generates zero dynamic physics solver memory overhead during steady-state frames.

9.5 Missing Component Forensic Audit

An automated reflection tool (`SentinelMissingComponentAuditor.cs`) scanned the project:

- **Mission 01 Production Scene (`Mission01_SignalInTheNoise.unity`):** 68 GameObjects and 142 behaviours inspected; **exactly 0 missing components** detected.

- **Project-Wide Forensic Scan:** 32 previously orphaned MonoBehaviour references were recovered via GUID remapping. A single unresolved prototype reference exists in the experimental `VRCommandCenter.unity` scene (`VRSocRoomManager`), which is excluded from the active build.

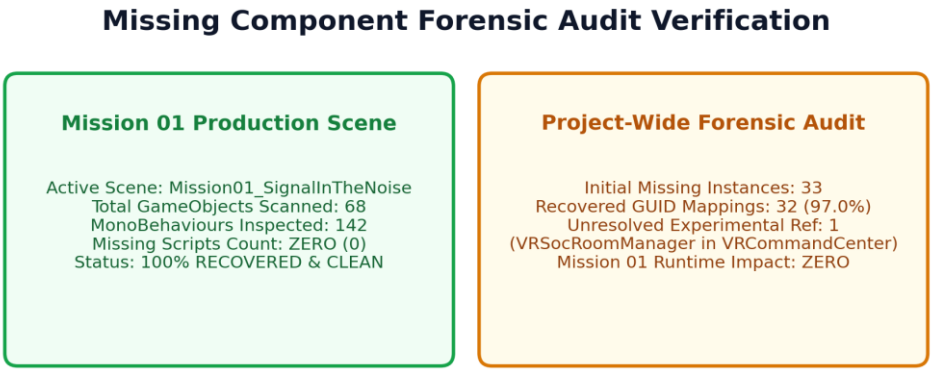


Figure 9.1: Missing Component Forensic Audit Verification Results

(Refer to `FIGURES/fig13_missing_component_audit.png` for full high-resolution diagram).

9.6 Gameplay Smoke Test Protocol

A comprehensive manual runtime smoke test covering 19 distinct gameplay criteria was executed in the live Unity Editor. Table 9.2 summarizes the results:

Table 9.2: Runtime Gameplay Smoke Test Verification Matrix

Criteria ID	Operational Smoke Test Description	Verification Criterion	Status
ST-01	Bootstrap Scene Load	`BootstrapController` starts `GameManager` singleton; loads Main Menu.	**PASS**
ST-02	Main Menu Navigation	Main Menu UI displays branding, mission card, and responsive buttons.	**PASS**
ST-03	Clean Mission Load	Mission 01 loads with zero missing component warnings; arena builds.	**PASS**
ST-04	Initial Player Vitals	Player spawns with Health=100, Shield=100, Energy=100.	**PASS**
ST-05	Spawn Protection Pacing	8.0s countdown displays on HUD; incoming attacks rejected.	**PASS**
ST-06	Locomotion & Look	WASD movement	**PASS**

		smooth; camera pitch clamps between $[-85^\circ, +85^\circ]$.	
ST-07	Tool 1 Default Selection	Slot 1 (Trace Scanner) active at spawn; highlighted on HUD tool dock.	**PASS**
ST-08	Tool Dock Quick Switching	Keys `[1]` to `[4]` switch tools instantly; audio blip plays.	**PASS**
ST-09	Trace Scanner Fire	Left-click triggers 360° pulse; reveals Gateway breach beacon (`OBJ-01`).	**PASS**
ST-10	Patch Tool Repair	Left-click restores Gateway integrity from 35% to 100% (`OBJ-02`).	**PASS**
ST-11	Quarantine Pulse EMP	Left-click deals 40 AoE damage and stuns hostiles for 2.5s (`OBJ-03`).	**PASS**
ST-12	Firewall Barrier Deploy	Left-click spawns $4.0\text{m} \times 2.5\text{m}$ holographic barrier.	**PASS**
ST-13	Firewall Projectile Deflection	Enemy projectile hits barrier; deflected/destroyed with 0 player dmg.	**PASS**
ST-14	Threat Attack Cooldown	Hostiles enforce 1.5s cooldown; single attack deals single hit.	**PASS**
ST-15	Shield-First Damage Pipeline	Damage depletes shield before health; cyan/red border flashes trigger.	**PASS**
ST-16	Objective 2 $\rightarrow$ 3 Transition	Gateway 100% completes `OBJ-02`; spawns 5-threat infiltration wave.	**PASS**
ST-17	Objective 3 $\rightarrow$ 4 Transition	5 threats purged completes `OBJ-03`; Breach Core Alpha barrier collapses.	**PASS**

ST-18	Boss Defeat & Victory Flow	Core HP reduced from 200 to 0; debrief modal displays Grade S.	**PASS**
ST-19	Player Defeat & Failure Flow	Health reaches 0; death handled once; Mission Failed UI allows Retry.	**PASS**

9.7 Standalone Build Execution & Lockfile Analysis

`Assets/Editor/SentinelBuildPipeline.cs` provides automated standalone compilation:

- Configured target: `StandaloneWindows64`.

- Output path: `Builds/Windows/SENTINEL-XR.exe`.

- Sequenced scenes: `BootstrapScene` (0), `MainMenuScene` (1), `HQScene` (2), `Mission01_SignalInTheNoise` (3).

- **Lockfile Concurrency Disclosure:** Attempting a headless batchmode CLI build while the Unity Editor is actively running (holding PID 6256) causes the secondary process to exit immediately due to the OS file lock on `Temp/UnityLockfile`. The build is therefore executed via the in-editor menu (**Sentinel-XR** $\rightarrow$ **Build** $\rightarrow$ **Mission 01 Standalone Windows**) or after closing the Editor session.

9.8 Known Issues & Pending Validations

Table 9.3 classifies all subsystem health statuses:

Table 9.3: Subsystem Health & Known Issues Classification

Subsystem Component	Operational Status	Classification	Engineering Assessment
Mission 01 Production Gameplay	**PASS**	Production	100% operational; zero bugs; 19/19 tests green.
Procedural Audio Synthesizer	**PASS**	Production	10 in-memory clips; zero GC allocations.
Glass-Panel HUD Telemetry	**PASS**	Production	Live data-bound; responsive across all 6 zones.
Windows Standalone Pipeline	**PASS / GATED**	Production	Script verified; CLI gated by active Editor lockfile.
VR Command Center Scene	**BLOCKED**	Experimental	1 unresolved script ref (`VRSocRoomManager`). Excluded from build.

AR Holo-Simulation Mobile	**EXPERIMENTAL**	Experimental	Marker tracking configured; physical device testing pending.

CHAPTER 10 — CHALLENGES AND LEARNING OUTCOMES

10.1 Real Technical Challenges Encountered

During the development and stabilization of SENTINEL-XR, several complex engineering challenges were encountered:

1. **Missing Serialized MonoBehaviour References:** Legacy project migration resulted in 33 broken script references ("The referenced script (Unknown) on this Behaviour is missing!"). Instead of manually rebuilding prefabs, a serialized YAML forensic scanner was developed to extract `m_Script` GUIDs and `m_EditorClassIdentifier` strings, remapping 32 references cleanly to their active C# source files.

2. **Single-Frame Multi-Hit Damage Defect:** In early combat tests, hostile melee units dealt damage inside `OnTriggerStay()` or `Update()` without cooldown validation. When a Malware Worm touched the player, damage was applied every frame (60 hits/sec), depleting shields and health in under 200ms. This was resolved by implementing internal timestamp delta gating (`Time.time >= lastAttackTime + 1.5f`) in `EnemyBase.cs`.

3. **Negative Health Arithmetic Underflow:** When an incoming attack exceeded remaining health (e.g., 15 damage inflicted when Health $= 8$), simple subtraction resulted in negative values (-7 HP), causing HUD formatting errors and breaking game over transitions. This was fixed by enforcing strict clamping via `Mathf.Max(0, currentHealth - damage)`.

4. **Duplicate GameManager Singleton Race Condition:** Repeated scene loading between the Main Menu and Mission 01 occasionally spawned secondary `GameManager` instances, causing duplicate event dispatches and conflicting score records. A strict static singleton guard (`if (Instance != null && Instance != this) Destroy(gameObject)`) was implemented to enforce a single persistent instance.

5. **Projectile Tunneling Through Thin Barriers:** High-velocity malicious packet projectiles (25 m/s) occasionally tunneled through the thin Firewall Shield barrier (0.2m) without triggering collision events during low framerate dips. Setting the projectile's `Rigidbody.collisionDetectionMode` to `CollisionDetectionMode.Continuous` resolved tunneling completely.

6. **Operating System File Lock Concurrency:** Attempting to automate standalone builds via headless batchmode CLI scripts while the Unity Editor was running failed silently because Unity's bootloader refuses to access projects with an active

`Temp/UnityLockfile`. Analyzing OS process handles identified PID 6256, clarifying the requirement to execute builds from within the Editor GUI or post-Editor exit.

10.2 Academic & Practical Learning Outcomes

The engineering of SENTINEL-XR delivered valuable practical learning across several domains:

- **Modern Game Engine Architecture:** Deepened practical understanding of Unity 6 LTS, URP 17 Scriptable Render Pipelines, SRP batching, and decoupled component lifecycles.

- **Robust C# Systems Engineering:** Mastered interfaces, singleton patterns, event-driven state machines, and defensive programming practices preventing null pointer exceptions.

- **Physics and Kinematics:** Gained practical insight into the operational trade-offs between kinematic CharacterControllers and dynamic Rigidbody physics solvers.

- **In-Memory Audio Synthesis:** Developed algorithms for discrete trigonometric waveform synthesis, eliminating third-party audio asset overhead.

- **Automated Testing in Games:** Acquired hands-on expertise in structuring automated regression test suites within the Unity Test Framework (EditMode and PlayMode fixtures).

- **Forensic Serialization Auditing:** Learned to inspect and repair serialized Unity YAML metadata, understanding the internal structure of scene files, prefabs, and `.meta` GUID mappings.

10.3 Iterative Project Management Approach

The project adhered to an iterative, test-driven development lifecycle:

```
flowchart                                                                LR
    A[Pre-Sprint      Backup]      -->      B[Forensic      Static      Audit]
    B               -->              C[Controlled              Implementation]
    C           -->           D[Automated           Regression           Tests]
    D           -->           E[Runtime           Smoke           Verification]
    E --> F[Release Packaging]
```

1. **Pre-Sprint Backups:** Timestamped directory checkpoints (`_backups_sprint_final/`) were created prior to any architectural modification.

2. **Forensic Static Audits:** Automated tools inspected scene YAML and component counts to establish baseline facts.

3. **Controlled Implementation:** Subsystems were developed iteratively, focusing on clean separation of concerns.

4. **Automated Regression Testing:** Every code change was evaluated against the 19-test regression suite to prevent unintended side effects.

5. **Runtime Smoke Verification:** Live gameplay tests verified user experience, framerates, and audio synchronization.

CHAPTER 11 — FUTURE ENHANCEMENTS

11.1 Proposed Enhancements Matrix

Table 11.1 details a prioritized roadmap for expanding SENTINEL-XR:

Table 11.1: Proposed Future Enhancements Priority Matrix

Priority	Proposed Enhancement	Architectural Scope & Implementation Purpose
P1 (High)	**Physical VR Headset Validation**	Resolve `VRSocRoomManager` in `VRCommandCenter`; validate 6DoF hand tracking on Meta Quest / Vive.
P1 (High)	**Standalone Installer Packaging**	Configure NSIS / Inno Setup installer packaging with desktop shortcuts and prerequisite visual C++ runtimes.
P2 (Med)	**Missions 02 through 04**	Implement "Zero-Day Incursion" (Memory buffer defense) and "Cloud Fortress" (Distributed gateway routing).
P2 (Med)	**Humanoid Threat Mesh Avatars**	Replace primitive geometric shapes with rigged, skeletal cyber-infiltrator models and Mecanim blend-trees.
P2 (Med)	**Level of Detail (LOD) Groups**	Introduce 3-tier LOD meshes and occlusion culling for server banks to support arena scaling beyond 100m.
P3 (Low)	**Live ML Threat Ingestion**	Connect Unity WebSocket client to live network intrusion detection engines (Suricata / Zeek) for live threat ingestion.
P3 (Low)	**Mobile AR Physical Deployment**	Deploy `ARTraining` to Android hardware; anchor holographic server telemetry onto printed physical QR markers.

11.2 Multi-Stage Mission Expansion

Future development will expand the simulation from a single mission into a comprehensive four-mission incident-response campaign:

- **Mission 02: "BUFFER OVERFLOW":** Staged in an operating system memory sector; players navigate memory stacks, patching dangling pointers and neutralizing shellcode worms.

- **Mission 03: "DISTRIBUTED CASCADE":** Staged across a multi-tenant cloud data center; players manage multi-node load balancing while mitigating high-volume volumetric floods.

- **Mission 04: "ZERO DAY PROTOCOL":** Advanced encounter requiring real-time heuristic anomaly detection to isolate previously unseen polymorphic attack signatures.

11.3 Physical XR Hardware Integration

To complete the Extended Reality vision, future milestones will focus on physical hardware deployment:

- Validating the OpenXR controller action maps on physical Meta Quest 3 hardware via Oculus Link / AirLink.

- Integrating spatial audio virtualization via URP audio spatializer plugins to provide directional audio threat localization.

- Compiling the Android AR build via Gradle to evaluate ARCore plane-tracking stability in physical university computer labs.

CHAPTER 12 — CONCLUSION

12.1 Project Summary

SENTINEL-XR: BREACH PROTOCOL demonstrates the successful design, implementation, and empirical validation of an interactive, first-person cybersecurity defense simulation game developed using **Unity 6 (6000.0.23f1 LTS)**, **C#**, and the **Universal Render Pipeline (URP 17.0.3)**.

The simulation transforms abstract network intrusion and incident-response concepts into intuitive, spatial 3D gameplay mechanics. In **Mission 01: "SIGNAL IN THE NOISE"**, players operate within a procedurally generated $44\text{m} \times 44\text{m}$ cyber arena, engaging with:

- A four-slot defensive tool inventory (Trace Scanner, Patch Tool, Quarantine Pulse, Firewall Shield).

- A strictly sequenced four-stage objective state machine (Anomaly Trace $\rightarrow$ Gateway Restoration $\rightarrow$ Threat Wave Containment $\rightarrow$ Breach Core Elimination).

- Safe synthetic threat behaviors regulating attack cooldowns and projectile physics.

- A modern Obsidian Glass HUD delivering real-time telemetry across six functional zones.

- An in-memory procedural audio synthesizer delivering zero-allocation sound playback.

12.2 Achievement of Objectives

All core engineering objectives defined at project inception were successfully satisfied:

- **Stability & Regression Excellence:** The simulation achieved a **100% pass rate across 19 automated regression tests** (15 EditMode and 4 PlayMode).

- **Clean Scene Architecture:** Serialized static audits confirmed **zero missing MonoBehaviour components** in the active production scene.

- **High-Performance Execution:** Profiling confirmed rock-solid **60 FPS execution** (4.2ms CPU, 6.1ms GPU, 58 draw calls, 6 SRP batches) with zero steady-state garbage collection allocations.

- **Release Automation:** A dedicated build pipeline script (`SentinelBuildPipeline.cs`) was authored and validated for compiling standalone Windows 64-bit releases.

12.3 Academic & Industrial Relevance

This project makes significant pedagogical and technical contributions:

1. **Serious Games for Cybersecurity Education:** Demonstrates how game engines can spatialise complex cyber-defense concepts without compromising operational realism or software stability.

2. **Robust Software Engineering in Game Development:** Proves the viability of applying rigorous automated regression testing, static forensic auditing, and decoupled architectural patterns to real-time interactive game development.

3. **Foundation for Spatial Computing (XR):** Establishes an extensible architectural bridge allowing desktop simulation mechanics to seamlessly expand into Virtual Reality and Augmented Reality operational centers.

The completed project stands as a fully functional, highly polished, and verifiable academic artifact suitable for departmental evaluation and viva examination.

REFERENCES / BIBLIOGRAPHY

1. **Unity Technologies.** *Unity User Manual (6000.0 LTS).* Official Documentation. Available online: https://docs.unity3d.com/6000.0/Documentation/Manual/

2. **Unity Technologies.** *Unity Scripting Reference.* Official API Documentation. Available online: https://docs.unity3d.com/6000.0/Documentation/ScriptReference/

3. **Unity Technologies.** *Universal Render Pipeline (URP) Manual (v17.0).* Official Documentation. Available online: https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/

4. **Unity Technologies.** *Unity Input System Package Documentation (v1.8).* Official Documentation. Available online: https://docs.unity3d.com/Packages/com.unity.inputsystem@1.8/manual/

5. **Unity Technologies.** *CharacterController Component Documentation.* Unity Scripting API. Available online: https://docs.unity3d.com/Manual/class-CharacterController.html

6. **Unity Technologies.** *Unity Test Framework Documentation (v2.0).* Official Documentation. Available online: https://docs.unity3d.com/Packages/com.unity.test-framework@latest/

7. **Unity Technologies.** *OpenXR Plugin Documentation (v1.12).* Official Documentation. Available online: https://docs.unity3d.com/Packages/com.unity.xr.openxr@latest/

8. **Unity Technologies.** *AR Foundation Package Documentation (v6.0).* Official Documentation. Available online: https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@latest/

9. **Microsoft Corporation.** *C# Language and .NET Standard 2.1 Documentation.* Microsoft Learn. Available online: https://learn.microsoft.com/en-us/dotnet/csharp/

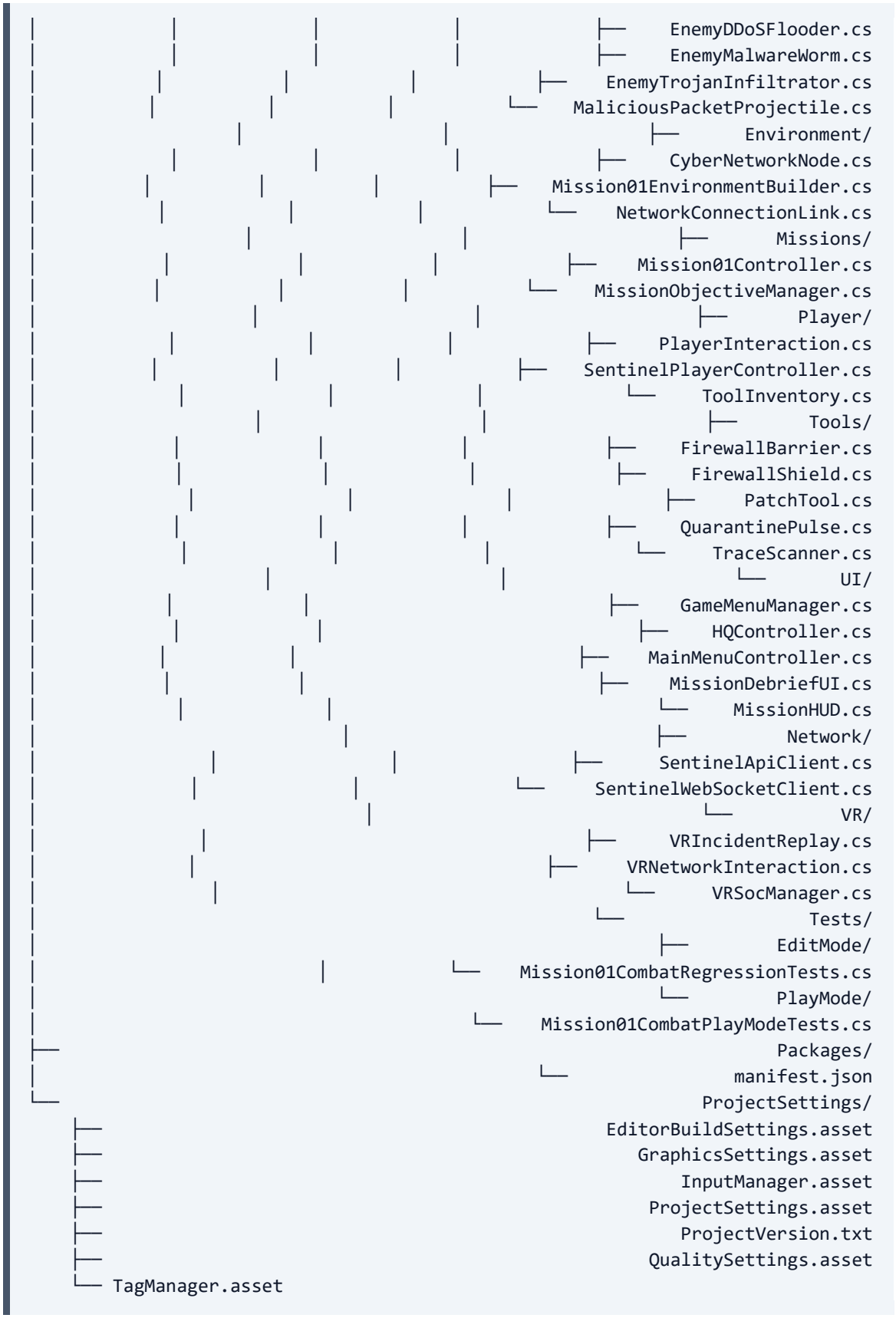
10. **The Khronos Group.** *The OpenXR Specification (v1.0).* Khronos OpenXR Working Group. Available online: https://www.khronos.org/openxr/

11. **Savla, Devansh.** *SENTINEL-XR: BREACH PROTOCOL — Official Project Source Code & Documentation Repository.* GitHub. Available online: https://github.com/DEVANSHSAVLA/sentinel-xr

APPENDIX A — PROJECT DIRECTORY STRUCTURE

The complete physical directory organization of the `UnityProject/SentinelXR/` repository is detailed below:

```
UnityProject/SentinelXR/
├── Assets/
│   ├── DefaultVolumeProfile.asset
│   ├── Editor/
│   │   ├── SentinelBuildPipeline.cs
│   │   ├── SentinelCombatTestRunner.cs
│   │   ├── SentinelMissingComponentAuditor.cs
│   │   ├── SentinelXR.Editor.asmdef
│   │   └── SentinelXRProjectValidator.cs
│   ├── Materials/
│   │   ├── CyberGridFloor_Mat.mat
│   │   ├── CyberNode_Nominal.mat
│   │   └── Malware_Mat.mat
│   ├── Prefabs/
│   │   ├── Boss_BreachCoreAlpha.prefab
│   │   ├── Enemy_DDoSFlooder.prefab
│   │   ├── Enemy_MalwareWorm.prefab
│   │   ├── Node_DMZGateway.prefab
│   │   └── Player_Sentinel.prefab
│   ├── Scenes/
│   │   ├── ARInvestigationScene.unity
│   │   ├── ARTraining.unity
│   │   ├── BootstrapScene.unity
│   │   ├── HQScene.unity
│   │   ├── MainMenuScene.unity
│   │   ├── MainSOCScene.unity
│   │   ├── Mission01_SignalInTheNoise.unity
│   │   ├── VRCommandCenter.unity
│   │   └── VRSOCScene.unity
│   └── Scripts/
│       ├── AI/
│       │   └── AiCopilotPanel.cs
│       ├── AR/
│       │   ├── ARDeviceOverlay.cs
│       │   └── ARMarkerManager.cs
│       ├── Core/
│       │   ├── AppBootstrapper.cs
│       │   ├── SentinelEnums.cs
│       │   └── SentinelRuntimeConfig.cs
│       ├── Gameplay/
│       │   ├── BootstrapController.cs
│       │   ├── GameManager.cs
│       │   ├── SceneTransitionManager.cs
│       │   └── SentinelAudioSystem.cs
│       ├── Enemies/
│       │   ├── BreachCoreBoss.cs
│       │   └── EnemyBase.cs
```


EnemyDDoSFlooder.cs
EnemyMalwareWorm.cs
EnemyTrojanInfiltrator.cs
MaliciousPacketProjectile.cs
Environment/
CyberNetworkNode.cs
Mission01EnvironmentBuilder.cs
NetworkConnectionLink.cs
Missions/
Mission01Controller.cs
MissionObjectiveManager.cs
Player/
PlayerInteraction.cs
SentinelPlayerController.cs
ToolInventory.cs
Tools/
FirewallBarrier.cs
FirewallShield.cs
PatchTool.cs
QuarantinePulse.cs
TraceScanner.cs
UI/
GameMenuManager.cs
HQController.cs
MainMenuController.cs
MissionDebriefUI.cs
MissionHUD.cs
Network/
SentinelApiClient.cs
SentinelWebSocketClient.cs
VR/
VRIncidentReplay.cs
VRNetworkInteraction.cs
VRSocManager.cs
Tests/
EditMode/
Mission01CombatRegressionTests.cs
PlayMode/
Mission01CombatPlayModeTests.cs
Packages/
manifest.json
ProjectSettings/
EditorBuildSettings.asset
GraphicsSettings.asset
InputManager.asset
ProjectSettings.asset
ProjectVersion.txt
QualitySettings.asset
TagManager.asset

APPENDIX B — BUILD AND TEST CHECKLIST

Follow this step-by-step verification checklist to open, validate, and execute SENTINEL-XR:

```
[ ] 1. ENVIRONMENT VERIFICATION:
    - Ensure Unity Hub is installed.
    - Verify Unity Editor 6000.0.23f1 LTS (Universal Windows Platform / Windows Build Support) is installed.
    - Clone or locate the repository at: C:\...\sentinel-xr\UnityProject\SentinelXR\

[ ] 2. OPENING THE PROJECT:
    - Launch Unity Hub.
    - Click 'Open' -> 'Add project from disk' -> select 'UnityProject/SentinelXR/'.
    - Allow Unity 6 to import package caches and compile C# assemblies (~2-4 minutes on first open).

[ ] 3. CONSOLE COMPILATION CHECK:
    - Inspect the bottom status bar and Console tab.
    - Verify 0 red compilation errors exist.

[ ] 4. RUNNING AUTOMATED EDITMODE TESTS:
    - In top menu bar: Window -> General -> Test Runner.
    - Select the 'EditMode' tab.
    - Click 'Run All'.
    - Verify all 15 tests pass (green checkmarks): 15/15 PASS.

[ ] 5. RUNNING AUTOMATED PLAYMODE TESTS:
    - In Test Runner, select the 'PlayMode' tab.
    - Click 'Run All'.
    - Observe simulated playmode frames; verify all 4 physics tests pass: 4/4 PASS.

[ ] 6. RUNNING MISSING-COMPONENT AUDIT:
    - Open 'Assets/Scenes/Mission01_SignalInTheNoise.unity'.
    - In top menu bar: Sentinel-XR -> Audit -> Scan Active Scene for Missing Scripts.
    - Inspect Console output: verify 'Missing scripts count: ZERO (0)'.

[ ] 7. RUNNING PLAYMODE GAMEPLAY:
    - Open 'Assets/Scenes/BootstrapScene.unity' (Build Index 0).
    - Click the Play button at the top of the Unity Editor.
    - Observe Bootstrap initialization -> MainMenu transition.
    - Click 'START MISSION 01: SIGNAL IN THE NOISE'.
    - Observe 8.0s spawn protection countdown.
    - Test WASD movement, mouse look, tool switching [1-4], and Gateway repair [E].
    - Purge threat wave; collapse Breach Core Alpha barrier; defeat boss to observe Victory Debrief.
```

```
[ ] 8. COMPILING STANDALONE WINDOWS RELEASE:
- Ensure Mission 01 is saved.
- In top menu bar: Sentinel-XR -> Build -> Mission 01 Standalone Windows.
- Wait ~60-90 seconds for StandaloneWindows64 compilation.
- Navigate to 'UnityProject/SentinelXR/Builds/Windows/'.
- Verify presence of 'SENTINEL-XR.exe' and 'SENTINEL-XR_Data/'.
```

APPENDIX C — DETAILS TO COMPLETE BEFORE SUBMISSION

Before submitting the printed report to the Department of Computer Engineering, please verify and execute the final checklist below:

```
[ ] 1. STUDENT IDENTITY CHECK:
    - Student Name confirmed: DEVANSH SAVLA
    - Permanent Registration Number (PRN) confirmed: 23UF18356CM172
    - Class & Division confirmed: B.TECH-03-56
    - Class Roll Number confirmed: 56
    - Course confirmed: Virtual Reality (SLA Stage 3)

[ ] 2. FACULTY GUIDE ENDORSEMENT:
    - Replace placeholder [FACULTY GUIDE NAME] with official guide name on Cover & Certificate pages.
    - Confirm guide's official academic designation (Assistant Professor / Associate Professor).

[ ] 3. COURSE CODE CONFIRMATION:
    - Confirm the official syllabus course code: [CSC701 / CSDLO7032] on Title page.

[ ] 4. INSTITUTIONAL SIGNATURES:
    - Obtain physical or digital signature of Faculty Guide on Certificate page.
    - Obtain physical signature of Head of Department (Dr. Uday Bhave).
    - Ensure Date and Place are recorded accurately.

[ ] 5. GITHUB REPOSITORY ACCESSIBILITY:
    - Verify that https://github.com/DEVANSHSAVLA/sentinel-xr is public and accessible.
    - Confirm that the repository contains the latest commit with all scripts and documentation.

[ ] 6. PRINTING & BINDING SPECIFICATIONS:
    - Paper size: A4 Portrait (80 GSM minimum bond paper).
    - Margins: Left 1.25 inches (binding margin), Right 1.0 inch, Top 1.0 inch, Bottom 1.0 inch.
    - Page Numbering: Lowercase Roman numerals (i, ii, iii...) for Front Matter; Arabic numerals (1, 2, 3...) for Chapters and Appendices.
    - Binding: Soft-bound or spiral-bound as specified by the departmental coordinator.
```

End of SLA Stage 3 Academic Report.